

Introdução à Programação

Para Iniciantes e Agências Web Modernas

ESCRITO POR

MILTON BOLONHA

EDIÇÃO 1.2.0

BRASIL

EDITORA BOOK4DEV

2024

Todos os direitos reservados

Introdução à Programação: Para Iniciantes e Agências Web Modernas
por Milton Bolonha

Publicado por Editora BOOK4DEV

Ribeirão Preto – SP

Copyright © 2024 Milton Bolonha

Todos os direitos reservados. Este livro não deve ser reproduzido sem a permissão do autor, nem partes e nem na íntegra, exceto por formas previstas no ordenamento jurídico Brasileiro. Para permissões de uso contate: miltonbolonha@gmail.com

Capa por Milton Bolonha

Impresso no Brasil

Edição 1.2.0

Introdução À Programação

Para Iniciantes e Agências Web Modernas

O VERSIONAMENTO, PRÉ-MOLDADO DA INDÚSTRIA
DE TECNOLOGIA, PROGRAMAÇÃO MODERNA

ESCRITO POR

MILTON BOLONHA

EDIÇÃO 1.2.0

BRASIL

EDITORA BOOK4DEV

2024

Todos os direitos reservados

SUMÁRIO

Introdução	6
Resumo	8
Material Didático	8
Requisitos Básicos	9
Milton Bolonha	12
Panorama Geral de Desenvolvimento	19
O Que É Versionamento	31
O Que É Repositório Git?	39
O Básico do Desenvolvimento Web	46
Desenvolvimento Web Moderno	64
O Seu Primeiro Template	70
Arquitetura do Molde	82
Análise de Websites e SEO	100
Técnicas Avançadas	106
Dicas Úteis	119

AGRADECIMENTOS

Obrigado ao meu Eu criança, que resolveu em 1998 abrir o código-fonte de um website, copiou o seu conteúdo em um arquivo **HTML** no próprio computador e depois abriu o arquivo, vendo que ali, na página web resultada, se abria um mundo de possibilidades fantásticas.

Sou grato pela empresa **Envato** fazer os melhores tutoriais desde sempre e pelos **YouTubers** americanos e indianos. Sou grato pela **Microsoft** e seu “tentáculo” de versionamento a **GitHub**.

Obrigado a todxs os que deram vida ou inspiraram-me no decurso da escrita deste. Obrigado a todos os bons e maus patrões que pagaram meu salário para trabalhar nessa área. Obrigado a todos os meus clientes.

Obrigado Miltinho, meu filho, por sempre me inspirar.

Por fim, obrigado a você que está lendo este. Peço que compartilhe, fale sobre e presenteie um(@) amig(@) com este livro.

INTRODUÇÃO

Aprender o idioma para escrever um website ou um aplicativo, é mais importante do que aprender inglês.

A atividade de programar é feita por meio de um monte de páginas preenchidas com escritos em linguagem humana técnica, em forma de códigos. Por isso, podemos dizer que a programação desses sistemas é também um livro de códigos.

Esses escritos estão não em códigos estranhos e sem sentido. Pelo contrário, apesar de escritos em linguagem estrangeira para a maior parte do mundo, a linguagem técnica é bem humana.

Aliás, são três as ações que um computador faz: **processar informação, armazenar informação e memorizar informação**. E não se engane que são informações aleatórias, o mundo da tecnologia gira igual a qualquer setor industrial, em torno do que é mais rápido, efetivo e lucrativo. Por isso, a **Eugenia 3.0**.

A industrialização é coisa humana e tomou proporções globais nesta área. Neste momento, existem muitas maneiras de criar sistemas web, alguns mais complexos e outros mais simples.

A curadoria de códigos e dos melhores padrões de desenvolvimento web é algo que deve ser feito por alguém muito experiente, que esteja na área o tempo suficiente para saber com segurança quais tecnologias estarão em alta pelos próximos anos. A saúde de um projeto de tecnologia depende dessas escolhas.

Para tanto, as tecnologias escolhidas e referências que irei citar/**grifar** vão garantir a mais eficiente, mais rápida, mais saudável tecnologia do mercado. Aquelas com maior número de programadores e com a maior legibilidade do mercado, pois um projeto deve ser escrito com linguagem simples, de forma que todos possam entender.

Para expandir o seu aprendizado para um projeto real é que preparamos o nosso molde. Com ele, não será necessário começar do zero, pois a mesma plataforma que vamos usar é usada por empresas como Meta/Facebook, Notion, Twitch, Tik Tok, Walmart, Apple, Nike, Netflix, Uber e Starbucks.

Você terá a responsabilidade de pesquisar maiores definições de cada novo **Termo** e assistir diversos vídeos a respeito dos termos, vídeos gratuitos em inglês e português.

Alguns ótimos cursos podem ser encontrados em sites como a **Udemy**, por preços que chegam até a **R\$40,00**.

RESUMO

Você está pronto para dar um salto de realidade na sua carreira como desenvolvedor web e engenheiro de software?

Com este livro você aprenderá o básico do **Git**, desenvolvimento web moderno, padrão de documentação e padrões da comunidade.

Também terá acesso a um molde de aplicativo/website em forma de modelo “**GitHub Template**”, contendo tecnologia de alto valor, de ponta, usada atualmente no mais competitivo mercado de programação.

Foram centenas de milhares de dólares investido nesse molde, feito especialmente para você. Você receberá as indicações de como usar esse modelo.

MATERIAL DIDÁTICO

Ao adquirir um exemplar deste, garanta também o acesso ao material didático acessando o website indicado em capítulo futuro. Lá você encontrará os seguintes itens:

- Carta de boas-vindas;

- Imagens para perfil, cover, stories, papel de parede para desktop e mobile;
- 02 videoaulas;
- 04 e-books;
- 01 hacker-book;
- Modelo GitHub de website em Next.js;
- 01 livro da aventura:
Eugenia 3.0: As Crianças CTO's (*pdf*);
- 01 álbum de músicas instrumentais para estudo: **As Crianças CTO's** (disponível nas plataformas de *streaming*);
- WhatsApp Stickers.

Acesse o material didático por meio do website:

miltonbolonha.github.io/next-boilerplate/edu

Vamos agora iniciar a nossa jornada!

REQUISITOS BÁSICOS

Acesso à Internet: Para acessar o material on-line, usar o template e recursos, além de visualizar o resultado das tarefas.

Navegação na Web: Os participantes devem saber como usar um navegador da web para acessar websites, como o **GitHub.com**.

Saber criar uma conta através de e-mail: Os participantes precisarão de uma conta de e-mail e uma conta GitHub para interagir com os serviços virtuais e usar as tecnologias do curso.

Editor de Texto Básico: Uma compreensão mínima de como usar um editor de texto simples para fazer alterações em texto, como o Bloco de Notas ou *TextEdit*.

Download de Arquivos: Os participantes precisarão entender como baixar e salvar arquivos da web em seus computadores ou nuvem.

Conceito de Diretórios e Arquivos: É necessária uma noção básica de como os computadores organizam arquivos em pastas/diretórios, pois os códigos são armazenados em arquivos e pastas.

Upload de Imagens: Os participantes precisarão entender como fazer upload de imagens usando um website, navegador e as pastas e arquivos do usuário.

Interesse e Vontade de Aprender: Embora os requisitos técnicos sejam leves, a disposição para aprender e seguir instruções é fundamental. Ser muito curioso sobre cada tópico, cada nome diferente, pode impulsionar seus estudos.

Caderno de Anotações: É importante ter em mente que a atividade-chave do programador é escrever. Adquirir o hábito de fazer pequenas anotações, para isso você pode usar sistemas web como o **Notion** ou o **GitBook**. Eu gosto de anotar meus estudos escrevendo e desenhando em pequenos cadernos.

Última dica, estou programando há 26 anos e todos os dias estou aprendendo algo novo. E não somente coisas complexas e de super programadores. É bem comum que pessoas programadoras iniciantes sejam curiosas e acabam descobrindo novas formas bem mais legais de se fazer algo.

Portanto, aproveite esse momento, curta a jornada e tenha paciência consigo. Vai dar tudo certo!

MILTON BOLONHA

Oi, eu sou **Milton Bolonha**, sou designer e programador de websites/aplicativos desde 1998.

Como especialista faço a curadoria e implementação sustentável de códigos. Possuo conhecimento ímpar do mercado de tecnologia.

Atuo como gerente de tecnologia com proficiência no governo de documentos e na otimização de alcance publicitário. Direciono investimentos e tomo decisões estratégicas em marketing, publicidade e branding.

Tenho vasta experiência no mercado de tecnologia Brasileiro e global. Sou engenheiro de software, de inteligência artificial, gestor de projeto **open-source** e ensino programação.

RESUMO PROFISSIONAL

Sou programador e designer desde 1998, filantropo desde 2007, autor best-seller pela **Amazon** desde 2018 e engenheiro de software "*Top Rated*" pela **UpWork** desde 2018.

Eu comecei a aprender a programar aos 11 anos de idade, isso era final da década de 90. Sempre fui autodidata e tive que desenvolver os meus próprios métodos de enxergar a programação web.

De lá pra cá foram literalmente milhares de websites, aplicativos e sistemas desenvolvidos. Meu movimento mais ousado na carreira, foi o de tentar pela primeira vez a vida na capital de São Paulo, saindo de um interior com 20 mil habitantes. E deu certo!

De São Paulo para muitos outros lugares. Passei também por crises financeiras globais, crises políticas constantes e devastadoras para essa área e por pandemia.

Sou professor voluntário em serviços filantrópicos, sempre amei fazer a diferença na vida das pessoas que me cercam.

Nesses anos todos, com uma carreira extensa na área de desenvolvimento web, eu tive a oportunidades tanto no Brasil como no exterior, tendo destaque como programador nacionalmente, nos Estados Unidos, Itália, Alemanha, Portugal e França.

Com sede em São Francisco, CA, EUA, eu faço parte do **GitHub Developer Program**, isso quer dizer que vou te dar todo o suporte **GitHub** que você precisa.

Por meio do programa **100 Open Startup**, fui contratado pela gigante da área da saúde **HapVida** para consultoria on-line em tecnologia.

Tive a minha startup **Edu4Dev**, avaliada pela **ACBOOST 22**, da **Associação Comercial de São Paulo**. A **Edu4Dev** se destacou com um produto e governança acima da média do programa.

Trabalhei por mais de 2 anos na agência web norte-americana **Green Hat Web Solution**. A Green Hat é uma agência especializada em **WordPress**.

A **Descola** é uma startup inovadora na área de cursos e tem como principal parceiro o **Cubo Itaú**. Eles usam em seu blog, tecnologia criada por mim, com os mesmos padrões utilizados neste livro.

Tenho longa carreira de serviços voluntários, iniciei essa paixão na Paraíba entre os anos de 2007 a 2009, com **trabalho filantropo** (não remunerado) em diversas cidades do estado.

TIMELINE PROFISSIONAL

Autônomo - Local e Mundo (on-line)

2006 ~ Atual - *CEO, CTO, Fullstack Developer, Sênior Designer, Arte Finalista, GitHub Developer, Arte Finalista, Editor de Livros, Diagramador, Autor Escritor, Filósofo, Produtor Musical, Compositor, Músico, Vocalista e Filantropo.*

UpWork - Mundo (on-line: Itália, França, Estados Unidos e Alemanha)

2018 ~ Atual - *Fullstack Developer, Designer e Copywriter.*

Green Hat - Denver, CO, EUA (on-line)

2018 ~ 2020 – *WordPress Developer, Web Developer e Designer.*

Agência Inoltz - Salvador/BA

2014 ~ 2015 - *PHP e Fullstack Developer.*

Agência IdeiaON - São Paulo/SP

2011 ~ 2012 - *PHP, WordPress Developer e Fullstack Developer.*

Agência DIX - Ribeirão Preto/SP

2010 - *PHP e Fullstack Developer.*

Servicom - Ribeirão Preto/SP

2009 ~ 2010 - *Joomla Developer e Back-End Developer.*

Trabalho Humanitário Filantrópico - Paraíba

2007 ~ 2009

World Designer - Ribeirão Preto/SP

2006 ~ 2007 - *Flash, Dreamweaver e Front-End Developer.*

TESTEMUNHOS

Veja abaixo o que estão falando da minha tecnologia e do meu trabalho.

Esses são testemunhos reais dos meus clientes:

Esse cara é um rockstar do código. Eu recomendo! :) Também é uma pessoa muito descontraída e comunicativa. Com certeza voltaremos a trabalhar com ele. CINCO ESTRELAS!

- Federico H., Milão, Itália.

O mercado está inundado de desenvolvedores de baixa experiência, mas Milton está claramente em um nível diferente, GOAT!

- Renato Q., Lisboa, Portugal.

"Milton é um desenvolvedor muito talentoso. Contratei-o há vários anos e conheço-o bem. As habilidades de Milton são muito altas. Eu recomendo ele." - **Ryan M., Denver, EUA.**

"Ótimo! Boa comunicação e trabalho rápido. Eu recomendo." - **Jordane P., França.**

"Milton nos ajudou a otimizar nosso site *Gatsby* e resolver nossos problemas de "*build*". Ótimo trabalho!" - **Nathan P., Richmond, EUA.**

"Milton é um excelente profissional e definitivamente um especialista em *Gatsby*. Ele possui boa qualidade de código e atenção aos prazos." - **Gustavo P., São Paulo, BR.**

"Desenvolvimento sólido. Obrigado!" - Emily W, Hollywood, **Los Angeles, EUA.**

"Milton é um ótimo designer e desenvolvedor. Ele é extremamente talentoso em WordPress, git e *Gatsby*. Eu recomendo muito o Milton." - **Ken Miller, Denver, EUA.**

"Desenvolvedor muito bom para trabalhar. Rápido, eficiente e conhecedor. Certamente recomendo e quero trabalhar novamente com ele." - **Gustavo O., Brasília, BR.**

PANORAMA GERAL DE DESENVOLVIMENTO

No mundo do desenvolvimento web, existem diversas áreas. Cada área exige habilidades únicas e conhecimentos específicos para atender às demandas internacionais e locais do mercado digital.

As pessoas desenvolvedoras mais experientes se tornam especialistas e o mais neófito é o júnior. Tendo essa escala em mente, é comum encontrar no mercado a seguinte hierarquia: júnior, pleno, sênior e especialista.

Toda empresa é uma empresa de tecnologia e precisa de um **Gerente de Tecnologia, C-Level (CTO)**.

Focando nos programadores, listamos abaixo algumas das carreiras e responsabilidades da pessoa desenvolvedora.

DEV, DEVELOPER, DESENVOLVEDOR, PROGRAMADOR

Os programadores se dividem naqueles que vão cuidar dos códigos que compõem a estética do website/app e os que vão lidar com informações mais

sensíveis e técnicas, chamados de **Programador Front-End** e **Back-End** respectivamente.

FRONT-END DEVELOPER

Os desenvolvedores **Front-End** são responsáveis por traduzir os conceitos de design em interfaces gráficas para o usuário, com elementos interativos e visualmente atraentes.

Esses programadores dominam tecnologias como **HTML**, **CSS** e **JavaScript** para criar layouts responsivos, animações suaves e interações com o cliente. Além disso, otimizam a experiência do usuário para diferentes dispositivos e navegadores, garantindo a acessibilidade e usabilidade do website ou aplicativo.

Atualmente, o **ReactJS** é a ferramenta mais usada pelos desenvolvedores **Front-end** (combinado com o **NextJS**). Esses profissionais também são os responsáveis por implementar sistemas para inclusão como o **ARIA**. Ainda podem ser especialistas em tecnologias como **WordPress** e **JAMStack**.

BACK-END DEVELOPER

Os desenvolvedores **Back-End** trabalham nos bastidores para criar as funcionalidades que os clientes

e administradores usam no **Front-End** e são responsáveis por interações com servidores.

Dependendo da extensão do seu trabalho, podem ter uma especialização que lhes garanta um título chamado de **DevOps** (desenvolvedor de operações).

O **Front-End** acessa funcionalidades privadas, como sistema de login, através do **Back-End** e é no **Back-End** que as funcionalidades para conteúdo sensível agem.

Por exemplo, um sistema **Back-End** que é o estoque de produtos, necessariamente precisa do **Front-End** onde o programador fez diversos componentes visuais para que cada produto possa ser apresentado no formato de página web.

O programador **Back-End** trabalha com diversas tecnologias, algumas delas são: **Javascript, NodeJS, NPM, Banco de Dados, NoSQL, SQL, GraphQL, Docker, PHP, Composer, Laravel, MVC, Nginx, Cpanel, Linux** e etc.

DESENVOLVEDOR WEB

Há alguns sistemas que são locais e não estão disponíveis na internet. Os programadores desses sistemas não são desenvolvedores web.

A pessoa desenvolvedora web que se torna completa, é a especialista em ferramentas específicas

para criação de páginas web. É um mix de conhecimentos que a torna uma profissional versátil e que combina partes das habilidades do **Front** e **Back-End**.

Essas pessoas projetam e desenvolvem websites completos, garantindo as funcionalidades desejadas tanto na parte administrativa, quanto no design e na performance.

São proficientes em tecnologias que levam os seguintes nomes: **CSS, SCSS, JQuery, Gulpy, ReactJS, Semântica HTML, FTP, Cpanel**, etc.

Esta categoria de pessoa desenvolvedora tem ganho muita relevância no mercado, uma vez que os navegadores web estão sendo cada vez mais utilizados para criar aplicações robustas.

FULL STACK DEVELOPER

O **Full Stack Developer** é um especialista multifacetado que domina todas as etapas do desenvolvimento **Back** e **Front-End**.

Esses super desenvolvedores são capazes de trabalhar tanto na interface do usuário, quanto na infraestrutura do servidor, gerenciando bancos de dados, rotas de navegação e rotas externas, ou **API's**.

A sua experiência e conhecimento permite que crie produtos digitais completos, desde o conceito inicial até a implementação final. Conhecem banco de dados, sistemas de teste, sistemas de gerenciamento de tarefas e diversas ferramentas avançadas. Ou seja, é o cara/ a mina que sabe fazer a coisa acontecerem e de forma correta.

A pessoa desenvolvedora fullstack developer ainda pode ter amplo conhecimento de ferramentas nativas ao mobile, como o **React Native**.

JAVASCRIPT E NODEJS DEVELOPER

Dizem que na ilha de **Java** o café produzido por lá é um café único e maravilhoso. Bebida essa muito consumida pela equipe de desenvolvimento da linguagem de programação **Java**. Anos mais tarde e com o advento da internet se popularizando no mundo, essa equipe fez uma linguagem mais simples, para a internet e baseada em **Roteiros (scripts)**, o **JavaScript**.

Então veio o gigantesco complexo de esteiras de eventos e administração de tarefas chamado **NodeJS**, que serve tanto ao **Front-End**, quanto ao **Back-End**.

O desenvolvedor **NodeJS** é um especialista que cria sistemas e arquiteturas modernas usando a linguagem de programação **JavaScript**, combinada com essa poderosa ferramenta chamada **NodeJS**.

É sob a importante camada do **NodeJS** que muitas tecnologias em **JavaScript** agem. Atualmente é uma das mais populares tecnologias entre empresas e desenvolvedores web.

Com o **Node** (Nó) é possível ainda criar sua própria **CLI** e acessar as ferramentas mais incríveis que o ser humano já criou na área de tecnologia.

WORDPRESS DEVELOPER

O **WordPress** é o nome da tecnologia responsável por administrar websites (**CMS**) e já foi a mais popular ferramenta administrativa na web por muitos anos.

O **WordPress** foi feito na linguagem de programação **PHP**, tanto no **Back-End**, quanto no **Front-End**. Outras tecnologias que você encontrará junto ao **WordPress**: **Banco de Dados MySQL**, **CPanel**, **Servidor Apache**, **dotFiles**, **JavaScript**, **HTML** e **CSS**.

O **WordPress** foi a maior tecnologia de websites da década passada. Atualmente o **WP** é uma plataforma ideal para ser usada como **Back-End** apenas, usando **WPGraphQL**, transformando assim o **WP** no chamado **headlessCMS**, ou gestor de conteúdo sem **Front-End**.

O desenvolvedor especializado em **WordPress** é um profissional que conhece profundamente essa plataforma de gerenciamento de conteúdo. Eles

desenvolvem temas personalizados, criam plugins e widgets. E manipulam o código **CSS**, **HTML**, **PHP**, **MySQL** e **JavaScript** para adaptar o sistema às necessidades específicas de cada cliente. E tão importante quanto isso tudo, eles fazem isso seguindo os padrões de escrita do **WordPress**. Algumas empresas usam uma camada a mais para programar em **WP** chamada **Laravel**.

Esses programadores são responsáveis por garantir a segurança do site, implementando medidas de proteção contra ameaças e vulnerabilidades. É requerido de um **WP developer** que ele conheça a maneira correta de fazer um tema filho.

Por ser uma tecnologia consolidada, possui uma ampla gama de desenvolvedores e produtos. O programador **WP** terá conhecimento dos principais **plugins WP**, tais como o **Yoast**, **ACF**, **CF7**, **Smush**, **JS Minify**, **WooCommerce**, **JetPack**, **Elementor**, **W3 Total Cache**, **DeferJS**, **Akismet**, **Wordfence**, **Slider Revolution**, **WP Rocket**, **OptinMonster**, **Clone Post**, **Classic Editor**, **Redirection**, **WP SMTP**, **AddToAny**, **Visual Composer**, **Gravity Forms**, **SearchWP**, **Easy Digital Downloads**, **REST API Plugin**, **WPGraphQL**, etc.

JAMSTACK DEVELOPER

A sigla "**JAM**" significa **JavaScript**, **API** e **Markup**. Ela foi criada para expor essas tecnologias tão básicas que tomaram grandes proporções na internet.

Desenvolvedores web modernos preferem arquiteturas que tem como principal tecnologia o **JavaScript**, **SCSS** e o **MarkDown**, que é de fácil aprendizagem, leve e potente.

Eles utilizam algumas ferramentas **Javascript**, tal como o **Next.js** e o **Gatsby.js**, para criar websites rápidos. O **GatsbyJS** e o **NextJS**, ambos possuem diversas funcionalidades, mas em sua essência são criadores de **Front-End**, com poderes para o **Back-End** também.

São também criadores de websites estáticos, isso significa que eles vão armazenar apenas arquivos simples como **HTML**, **JavaScript**, **CSS**, imagens e vídeos. Arquivos como por exemplo **PHP** e tecnologias como **WordPress** não se enquadram nessa categoria de desenvolvimento.

Por meio da técnica chamada **API** (aplicação), você pode acessar rotas externas para informações contidas em serviços terceirizados. As **API's** ganharam o mundo, atualmente servem banco de dados, ou até mesmo o sistema de login. Rotas externas são demais!

No **Wordpress**, usa-se os chamados **plugins** para a maioria desses serviços, mas até dentro do **WP** quando você precisa usar um **plugin** de vendas, por exemplo, como o do **Mercado Livre**, você estará fazendo uso da **API** do **ML**.

Destaca-se a técnica de acesso a informações via **API** chamadas **RestAPI** e o **GraphQL (Meta)**.

NEXT.JS DEVELOPER

O desenvolvedor **Next.js** utiliza essa ferramenta (um tipo de biblioteca para escrever páginas web). Com o **Next** você constrói websites e aplicações web que tem velocidade instantâneas, com uma experiência de usuário fluida e ótimo **SEO**.

O **NextJS** foi escrito em **JavaScript**, também usa a ferramenta **ReactJS**. Ele opera dentro do **NodeJS**.

É uma das ferramentas (**framework**) do **ReactJS** mais usada no mundo, confiada pelas maiores empresas do mundo. No Brasil não é diferente, muitas vagas pedem por jovens com experiência nesse framework.

O benefício de ter uma ferramenta como o **NextJS**, é a leveza e segurança do seu sistema, que roda em um servidor curiosamente chamado de “**serverless**”, que significa “**sem servidor/menos servidor**”.

Além de tudo, é livre, gratuito para usar e sem custo mensal.

É importante que o desenvolvedor **NextJS** use no mínimo a versão **13(Λ)** dessa ferramenta. É geralmente combinado com o servidor **Vercel** (donos do **NextJS**).

GATSBY.JS DEVELOPER

Similar ao **NextJS**, o **Gatsby.js (v5Λ)** é uma forma de se criar websites e aplicativos (**PWA**). São conhecidos pela alta performance e segurança. E assim como o **NextJS** também é livre, grátis e sem custo operacional.

Podemos montar um funil inteiro de vendas utilizando o **GatsbyJS**. É possível ligar seu website/app com **WhatsApp**, **Inteligência Artificial**, envio de e-mails, coleta de dados de dispositivos sensoriais e etc.

Trabalhar com a versão **GatsbyJS 5**, ou superior (Λ), é muito importante e é obrigatório para os desenvolvedores **Gatsby**. Muitos recursos e mudanças vieram com essa versão.

Por fim, mas não menos importante, essa ferramenta poderosa pode ser integrada ao **GraphQL**. Imagine um cardápio onde todas as informações estruturadas no **Backend**, estão a disposição do **Frontend Developer**. Isso é o que o **GraphQL** faz. Você monta as suas páginas de forma mais fácil.

O **GatsbyJS** é geralmente combinado com a empresa de hospedagem em servidores (**serverless**) **Netlify**, que recentemente adquiriram o **Gatsby**.

DESENVOLVEDOR EMPREENDEDOR

Em alguns casos, é possível encontrar no mercado o profissional empreendedor que é um desenvolvedor **Back-end**, mas também faz o **Front-end**, é especialista em SEO, em marketing digital, também é designer, faz brands, trabalha com impressos, é cientistas de dados, analisa métricas, escreve textos publicitários, faz postagens, é videomaker, faz anúncios, resolve bugs que ninguém resolve, faz *hacks* que ninguém consegue, mas todo mundo precisa, faz atendimento ao cliente, serve o café, limpa o escritório, faz almoço, fala inglês, trabalha com freelance e busca por contratos de longo termos. Resumindo.

GITHUB COPILOT IA

Apontada como o futuro da programação, a **Inteligência Artificial** da **GitHub**, a **GitHub Copilot** é uma ferramenta revolucionária para desenvolvedores que oferece suporte na criação de códigos.

A **Copilot** literalmente escreve códigos para o programador, ou qualquer um que souber manuseá-la. Essa ferramenta analisa e sugere automaticamente linhas de códigos enquanto uma pessoa programa.

A **GitHub Copilot** vem transformando a forma como os desenvolvedores escrevem código, agilizando o processo de desenvolvimento e oferecendo soluções úteis para problemas comuns de codificação.

O QUE É VERSIONAMENTO

Cada página, ou arquivo que está na web, se configurado, pode possuir uma ou mais versões de si. No versionamento, cada alteração de um arquivo versionado gera uma nova versão dele.



O versionamento é um sistema que serve para muitos propósitos, vamos conhecer essa tecnologia básica da pessoa profissional em programação.

O VERSIONAMENTO

Como dito anteriormente, muitas alterações em algo cria uma versão daquele algo. Versionamento, é quando há um sistema para gerir essas versões.

Na vida é possível perceber que temos versões de nós mesmos e estamos carregando essas versões com a gente através dos anos.

Podemos dizer que na escola o nosso comportamento tem uma versão, para isso versionamos para o modo escolar quando estamos em sala de aula.

Já nas versões de sistemas de computador, essas versões são numeradas. Você pode já ter se perguntado, porque em jogos ou programas de computador, você sempre vê uma numeração seguida da palavra “beta”, como essa “3.0.1-beta”. Esse número é um identificador, como um número de lote do produto.

No meu livro **Eugenia 3.0: As Crianças CTO's**, é comum no dia-a-dia dos personagens algo chamado de **Versionamento de Realidades**. Vamos ver um trecho onde a personagem **Celeste** explica para o personagem **Miltinho** sobre o assunto:

...

<**Celeste**> (...) no mundo do versionamento existem três degraus: o maior, o menor e o degrau da correção.

Os versionadores [da Terra atuam] nas [versões de] correções, mas raramente nos versionamentos menores. E são banidos de fazerem versionamentos maiores, eles não possuem essa autorização.

<Miltinho> Eu não gosto da versão atual do Brasil beta. Acho que temos que fazer um versionamento maior.

<Celeste> Isso vai acontecer e você estará vivo para testemunhar o versionamento e a criação do Brasil beta 2.0XX.

<Miltinho> Brasil beta, dois, ponto, zero, xis, xis?

...

No caso dos personagens acima, foi usado a versão 2 beta e suas alterações mudam os três caracteres do meio **0.X.X**, um caractere numérico e duas letras. A próxima versão pode ser a 1.X.X, ou então 1.X.1.

Você pode criar um padrão totalmente novo para fazer a marcação de uma versão, mas vamos nos ater a um padrão bem simples e comum. Nós faremos o uso da seguinte forma de anotação de versão: 9.9.9-tag.

Para **alterações maiores**, com maior volume de mudanças, como uma nova versão de um jogo, você usa o primeiro número (ex.: **1.0.0**).

Para **alterações menores**, como inserir uma função totalmente nova, ou corrigir um grande defeito no mesmo jogo, você usa o número do meio (ex.: **0.1.0**).

E para **alterações de correção**, no exemplo do jogo poderia indicar a correção de uma palavra errada no menu do jogo (ex.: 0.0.1).

Depois da numeração é possível, às vezes, encontrar essas *tags*: beta, lts, current e alpha. Indica a fase de produção, por exemplo se está em testes, ou se é a versão já lançada oficialmente.

O QUE É O GIT E O GITHUB?

O **Git** (lê-se *guíti*) é a tecnologia por trás do versionamento para códigos de aplicativos, sistemas e websites. O **Git** permite aos programadores controlarem todas as mudanças feitas em um projeto.

Os desenvolvedores podem trabalhar em equipe, cada um em sua própria versão do código, depois combinam tudo em uma única versão para completar o projeto.

Com o **Git** você cria uma versão oficial, uma de testes para o cliente, outra de testes para equipe, ou seja, você cria o ambiente de desenvolvimento que quiser, sem complicações. Se houver algum conflito de códigos, o **Git** te dá as opções para resolver os conflitos.

O **Git** é como um sistema de lote de um produto, sabe aquele número estranho em qualquer produto do supermercado? Então, ele guarda muitas informações

sobre a produção, por exemplo, de um molho de tomate. Informações como safra, profissionais responsáveis e até a qualidade do tomate. Com o **Git** é igual, só que com códigos.

Resumindo, o **Git** é a ferramenta número um do programador e da agência web moderna para criar websites e aplicativos de uma forma industrial.

O QUE É A MICROSOFT GITHUB?

A empresa **GitHub** é a maior central de versionamento do mundo, usado por milhões de desenvolvedores e empresas.

GitHub

Você sabe o que é um Poupatempo? É uma repartição pública para burocracia agilizada, certo? O Poupatempo pode ser explicado como um “hub/central” de serviços também.

Imagine um lugar onde existem diversas unidades do Poupatempo, seria chamado de um “hub/central de Poupatempo”. Um “hub” é uma central que agrega e distribui processos e documentos.

A **GitHub** é um “**Hub de Git's**”. É uma plataforma na nuvem que hospeda/guarda códigos e oferece o serviço de versionamento desses códigos, além de ferramentas avançadas para programadores, incluindo detecção automática de vulnerabilidade e qualidade de código.



Octocat – O Gato Polvo (*octopus cat*). O polvo é o animal símbolo do versionamento, pois é como se cada versão fosse um tentáculo.

É no **GitHub** em que as equipes de programadores criam e trabalham todos juntos, usando as diversas versões do código de um aplicativo ou website.

É nesse sistema que os desenvolvedores experimentam e mesclam com segurança as suas programações.

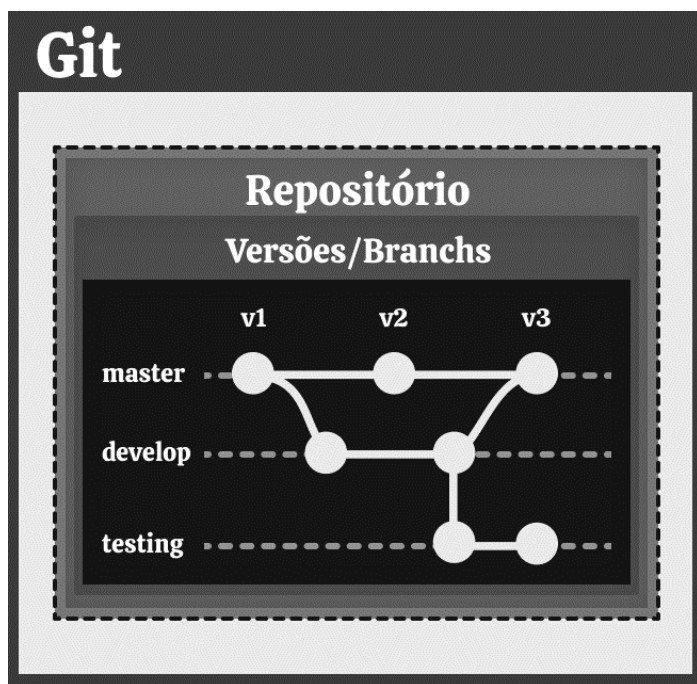
O **GitHub** também fornece ferramentas de comunicação, delegação de tarefas e automação para projetos.

É utilizado por diversos perfis, desde alunos aprendendo a programar até **CEO's** de empresas globais, possibilitando construir praticamente qualquer coisa no mundo.

GIT E VERSÕES

O **Git** é como o número de lote de um produto, contendo cada evento que aconteceu com aquele produto, só que no caso, são códigos.

A tecnologia **Git** permite que você crie versões do código e mantenha um fluxo de trabalho integrado com o modelo ágil.



Chamadas de **branches**, as versões levam também o nome de "**ramificações**". São clones do seu próprio código, que possuem todo histórico de mudanças de cada ramificação.

As versões mais comuns são:

- **master** (versão em produção)
- **staging** (versão de apresentação)
- **develop** (versão de desenvolvimento)

“Nunca” faça mudanças direto na master, use a versão de desenvolvimento e o chamado **Pull Request** para aprovação. Em um time, o seu superior direto irá aprovar ou reprovar e comentar o seu **PR**.

O QUE É REPOSITÓRIO GIT?

O **GitHub** vai armazenar o seu código em um **Repositório de Códigos**, que é basicamente o universo particular de cada projeto de programação.

Esses repositórios nos permitem voltar no tempo, como se tivéssemos uma máquina do tempo digital, para ver como tudo era antes de cada mudança feita.

Cada repositório possui sua própria história, configuração e ferramentas.

O REPOSITÓRIO GIT

O repositório de código possui diversos arquivos com programação. Se o universo fosse feito de códigos, digamos igual a um aplicativo, ele seria iniciado da seguinte maneira:

Em uma folha em branco estaria escrito **<IniciarUniverso />**. Essa folha seria colocada na esteira de eventos universais e o seu universo passaria por cada evento de pré construção e construção.

Então nosso repositório terá um arquivo contendo todas as informações para a esteira de eventos iniciar nosso aplicativo/universo.

A forma de iniciar qualquer coisa dentro do nosso universo seguiria o mesmo padrão acima, por exemplo:

...

```
<IniciarUniverso>
  <IniciarPlaneta />
</TerminarUniverso>
```

...

Note que podemos colocar coisas dentro do nosso universo. Agora estamos preparados para deixar o nosso universo mais interessante:

...

```
<IniciarUniverso>
  <IniciarPlaneta>
    <IniciarContinente>
      <IniciarPessoa falar="Olá, Brasil !" />
    </TerminarContinente>
  </TerminarPlaneta>
</TerminarUniverso>
```

...

Você percebeu o padrão de escrita usando os sinais de “*menor que*” e “*maior que*” (<>)?

Vamos ver outros tipos de padrões de escrita de códigos. No meu livro **Eugenia 3.0: As Crianças CTO's**,

eu usei diversos tipos de marcações encontrados na internet, veja:

...

**<Local geolocalização="desconhecida"
interior="sala-reunião">**

[CEO: "iBanco", "A Batalha pelo Sangue mudou o nosso país. Um milhão e 300 mil mortos. As cidades foram destruídas."]

(...)

#Senador A Presidenta autorizou a venda de Sangue por grandes empresas, mas o mercado será regulado pela estatal.

(...)

<Entrevistado emoção="empatia">Oi, me chamo Jão. Meu vizinho achou perto do Pátio do Colégio. E agora, com esse programa **<destaque>**Mais Sangue **</destaque>** eu vou poder criar um braço novo para o meu filho. **#topperson </Entrevistado>**

</Local>

...

Na aventura acima, eu usei o sinal de **#hashtag** e coloquei entre colchetes alguns personagens. Cada forma de escrita tem um significado. Dentro de alguns elementos eu coloquei os chamados “Atributos”, ex.:

<Elemento **atributo="verdadeiro" atributo2="XY9"** />

Eu ainda poderia ter usado um sinal de **@arroba** que nem do **Instagram**. Bem como deixar a fonte do texto escrito como no **WhatsApp** um pouco mais forte usando o ***bold***.

Muitas dessas marcações são feitas com sinais de maior ou menor (**<Elemento>**), colchetes, parênteses, chaves. Você vai precisar se acostumar com essa mentalidade.

Esses elementos estarão escritos em páginas, que estarão guardadas no repositório **Git**, que é como uma caixa embalada, que devemos despachar pelos correios, mas com poderes do versionamento. A cada vez que essa caixa é enviada para uma central (**hub**) de distribuição de versões, cria-se um histórico de todas as mudanças feitas lá dentro.

Caso você queira fazer mais mudanças, ou até reverter os códigos para uma versão antiga, você pegará outra caixa com o último código dentro, faz as suas alterações, fecha de novo e a envia para o serviço de versionamento **GitHub** por meio do **Git**.

MARCAÇÕES NO TEXTO

Eu te convido agora para uma nova fase do nosso aprendizado. Vamos falar sobre o tipo de escrita de códigos **Markdown**.

O CÓDIGO MARKDOWN E A CONVERSÃO EM HTML

O **Markdown** é a escrita para conversão de texto para linguagem usada em páginas da web (escritas no padrão **HTML**). É geralmente usado para registro de documentos simples, como uma postagem de website.

Você já percebeu que na web toda informação é escrita de forma diferente do que o usuário/cliente final vê na tela. Por exemplo, um texto simples é convertido na seguinte marcação:

`<h1>Título Aqui</h1>` (o “`<h1>`” é um código **HTML** que quer dizer Título mais importante)

O padrão **Markdown** ajuda a evitar que usuários finais precisem escrever código **HTML**, por isso dentro de um arquivo **MD** a frase acima seria escrita assim:

`# Título Aqui`

Ferramentas	Marcação	Exemplo
Títulos	#	# Texto
Parágrafos		Texto
Negrito	** **	Texto
Itálico	_ _	<i>Texto</i>
Citações	>	> Texto
Listas	-	- Texto
Links/Atalhos	[]	[Texto](https://link.com.br)
Imagens		![Texto](https://link.com "Texto 2")
HTML	<></>	Texto

Os padrões mais usados no **Markdown** são:

- **#** Título;
- **##** Título de importância 2;
- **###** Título de importância 3 (e assim por diante);
- Parágrafo (sem necessidade de marcação);
- ***** Lista (com asterisco na frente);
- **1.** Lista numerada;
- ****Reforçar/Bold****; **_Negrito_**;
- ****Itálico****, ou ***_itálico_***;
- **[Link](www.site.com)**;
- **![Texto da Imagem](www.site.com/img.jpg)**;
- ``código``, ou:

```
```  
Código
```
```
- Tabelas:
Exemplo | Valor
----- | -----

Exemplo 1 | R\$ 10

Exemplo 2 | R\$ 8

Dê uma olhada no arquivo **README.md** do repositório referência deste livro (**GitHub: miltonbolonha/next-boilerplate**).

🐉 O Profissional:

Olá Brasil!

****Milton Bolonha**** é especialista em tecnologia, fez o seu primeiro site em 1998 e hoje é considerado um dos melhores programadores do mundo pela ****UpWork****.

É autor best-seller pelo Kindle da Amazon com seus dois livros:

- ****Eugenia 3.0: As Crianças CTO's****
- ****Inteligências & Ordens - Descubra Quem É Você.****

O BÁSICO DO DESENVOLVIMENTO WEB

O que é a internet? A internet são as páginas web e seu conteúdo, armazenadas em computadores, que empresas ou pessoas adquiriram para expor os arquivos de um website ou app.

Para expor páginas web ao endereço virtual do seu website e ser encontrado por serviços de busca, esses computadores possuem endereços públicos, o famoso **www**.

Esse endereço público é registrado junto a órgãos locais e internacionais, como o **Registro.BR** que é responsável por indicar onde está cada página feita no mundo que tem no final a marca **“.BR”**.

Um programador deve fazer o registro do domínio para um website sempre registrando aquele domínio no nome e dados do cliente. Isso é muito importante! Quando o cliente te contratar, ele quer todas as ferramentas dele no nome dele e você também quer isso.

O NAVEGADOR

Existem vários tipos e camadas de serviços de buscas. Quando você digita '**www.dominio.com.br**', o **Navegador** age como um serviço de busca e faz uma consulta no sistema brasileiro de domínios (**Registro.br**).

Pode-se dizer, grosso modo, que todas as janelas são feitas de um elemento nativo do seu sistema operacional chamado "**Canvas/Tela de Pintura**".



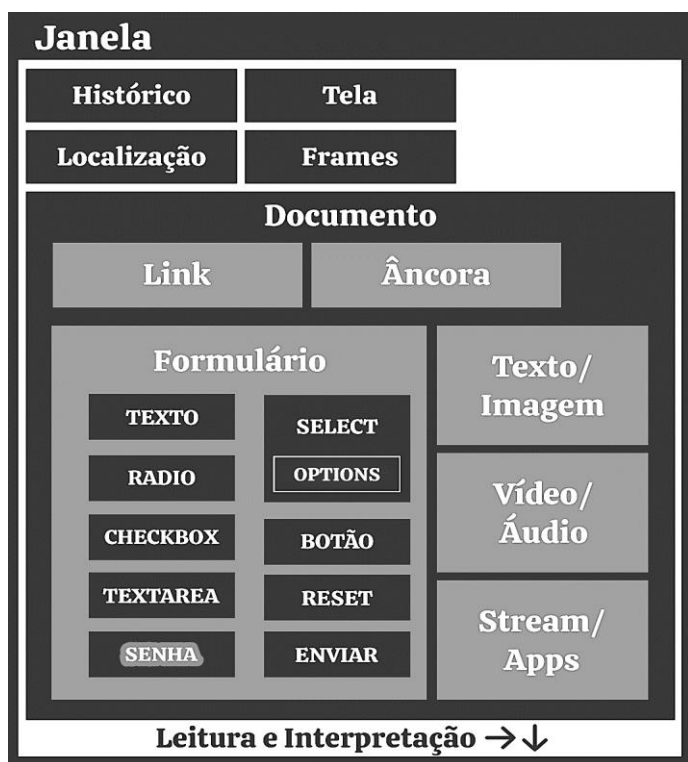
O navegador está apto a exibir **Documentos HTML**. Esses documentos têm acesso a janela do navegador, que dá diversas funcionalidades para o documento, tais

como: tamanho da tela, histórico de páginas visitadas, alertas do navegador e etc.

O DOCUMENTO WEB

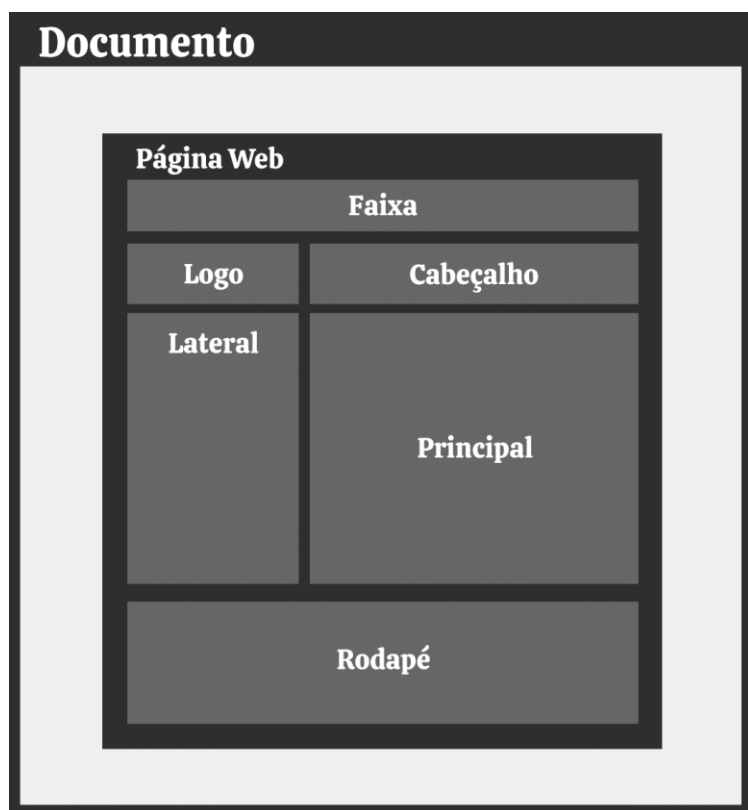
O **Documento HTML** é carregado, lido e interpretado pela janela do navegador, que possui algumas funções para ajudar a dar maior dinâmica à navegação.

Documento HTML é um arquivo que contém uma escrita para elementos visuais web (**<Elementos>**).



Uma página web apresentará o conteúdo que é de responsabilidade do desenvolvedor web inserir no **Front-End**.

Não é sua responsabilidade criar conteúdo, mas caso precise, ferramentas estilo **ChatGPT** podem ser úteis na hora do processo criativo.



Para montar um conteúdo o desenvolvedor usará **Elementos HTML**. O mesmo elemento que serve para um conteúdo em texto, é o elemento que serve para apresentar uma imagem, ou uma listagem em estilo

menu e até faixas coloridas para deixar o website mais bonito.

O **Elemento HTML** é um elemento híbrido, que aceita **texto, imagem, vídeo, áudio, svg** e outros formatos especiais, como **blob** e **iFrame**.

Esse **Elemento HTML** pode ser esticado pra baixo ou para direita; ele pode ser encolhido para a esquerda; o alinhamento do texto pode ser definido à direita, centro ou esquerda; o texto pode estar na direção padrão, ou invertido como um espelho; ele pode ser posicionado em um local determinado da página; e pode ganhar um identificador único, ou um identificador coletivo. Possui bordas, margens, espaçamentos, como veremos adiante.

Os **Elementos HTML** mais usados são o “divisor” (**<div>**) e o “parágrafo” (**<p>**). Outros elementos comuns são:

- **<html>** Essa é basicamente a forma que começa um texto em **HTML**;
- **<head>** Antes dos elementos visuais, as configurações da página ficam no topo do código, dentro do elemento “**cabeça**” (alusão ao começo do tempo de construção dos elementos **HTML**);
- **<body>** O corpo da página é onde estão os elementos visuais **HTML**.
- **<h1>** Título único com maior importância
- **** Importância dentro do parágrafo

- **
** Quebra de linha única
- **<a>** Tag âncora para um link
- **** Imagem
- **** Lista não ordenada
- **** Lista ordenada
- **** Item da lista
- **<table>** tabela

Cada elemento pode possuir atributos únicos e/ou gerais, tais como:

- ID, um registro geral de identificação;
- Classe, grupo ao qual pertence;
- Align, alinhamento do texto;
- Href, referências de links externos;
- Src (source), local onde está a imagem ou recurso;
- Largura e altura (width, height);
- Alt, para inclusão com textos alternativos à imagens;
- Title, título;
- Lang, idioma.

O BLOCO HTML

A grosso modo, um **Elemento HTML** é feito de diversas camadas de “**Canvas**”, uma sobrepondo a outra e servindo cada uma ao seu propósito, gerando assim o que eu chamo de **Bloco HTML**.

Cada elemento tem a largura da tela e uma altura de um simples parágrafo, formando assim uma linha, mas pode ser modificado por meio dos seus atributos.



As seguintes camadas do elemento, remete-nos a própria topologia matemática e de fronteiras:

- O elemento, a demarcação em si e seu conteúdo;
- Espaçamento (**padding**) ao redor;
- Borda é a fronteira do elemento, divide o que está dentro ou fora;
- Margem de distância entre qualquer outro elemento;

- Posição no documento.

Um **Documento HTML** possui coordenadas da sua página como um mapa. A tela inteira é utilizada para mapeamento de cada pixel no eixo **X** (largura/*width*) e **Y** (altura/*height*). Cada bloco HTML tem um posicionamento **XY** específico dentro desse mapa.

Quando escrevo códigos essas palavras saem de forma mecânica, mas é preciso atenção ao escrever de forma precisa, ou seja, perfeita para que funcionem.

Técnicas avançadas vão te avisar na hora que você escrever o código errado, bem como quando escrevemos no editor de texto uma palavra errada e o corretor nos ajuda na maioria das vezes.

DOM E BOM

○ **Document Object Model (DOM)** e o **Browser Object Model (BOM)** são essenciais no desenvolvimento web. Um é o **Documento HTML** e o outro é a estrutura para web do **Navegador**.

○ **DOM** representa a estrutura do **Documento HTML**, seus elementos, ex.: uma imagem sendo exposta, um parágrafo, ou título.

Enquanto o **BOM** abrange ações específicas do navegador, como histórico de navegação e janelas do navegador.

ÁRVORE DE DOM

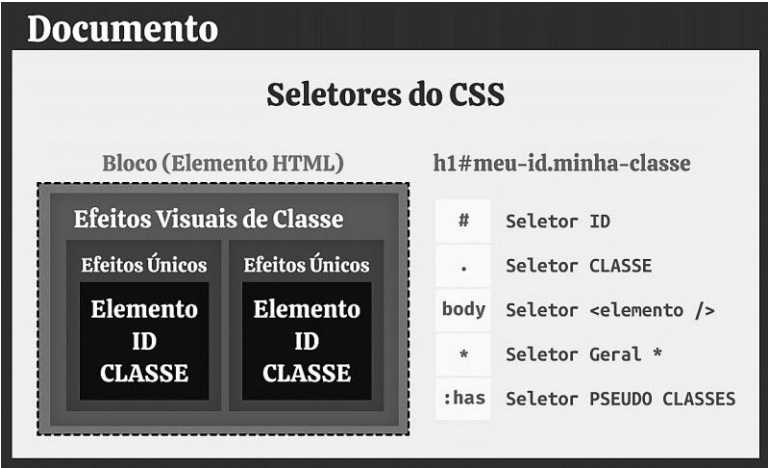
A **Árvore de DOM** é a hierarquia dos **Elementos HTML** que representam a estrutura do **Documento HTML**.

A **Árvore DOM** é feita dos elementos da página em uma estrutura de árvore (organograma), para facilitar a manipulação desses elementos, por meio da linguagem de programação **JavaScript**.

CSS E O DOM

Esses elementos (**DOM**) possuem atributos para serem estilizados, afim de ter um design bonito e prático. Para ser estilizado, o navegador faz uso de um arquivo criado pelo desenvolvedor **Front-End** que está escrito na linguagem de estilo **CSS**.

O **DOM** possui registros únicos e coletivos que o **CSS** usa para “pintar” cada elemento, são eles: **ID** e **Classe**.



Você levará anos para usar as mais diversas formas de selecionar um elemento, por meio do **CSS**. A essas formas dá-se o nome de **Seletores CSS**. Com eles, você será capaz de influenciar na aparência e no comportamento dos seus **Elementos HTML**.

O **ID** (identificação, tipo **RG**) é um indicador único, ou seja, somente é permitido que um Elemento HTML use esse **ID**. No **CSS** para selecionar um elemento por meio do **ID**, basta usar o símbolo **#** seguido pelo nome **ID** do elemento, ex.: **#meu-id**.

As classes são iniciadas com o ponto (**.**), ex.: **.minha-classe**.

O **Elemento HTML**, por sua vez, com os atributos **ID** e **classe** acima, ficará dessa maneira:

```
<h1 id="meu-id" class="minha-classe">Um Título  
Aqui</h1>
```

A seguir um exemplo de código em **CSS**:

```
h1#meu-id.minha-classe {  
  color: red;  
}
```

Abaixo o mesmo código escrito de outra forma:

```
#meu-id{  
  color: #ff0000;  
}
```

DOM VIRTUAL

Já sabemos que o **DOM** basicamente são os **Elementos HTML** do **Documento HTML**. Já o “**Virtual DOM**” é um **DOM** que só está disponível para quem usa a ferramenta criada pela **Meta** chamada **React.js**.

O **DOM Virtual** foi criado para ser uma representação do **DOM** original, ou seja, é uma cópia do **Elementos HTML** e serve para ser manipulado com alguns super poderes maiores que o navegador pode oferecer.

PACKAGES DOM

Existem algumas ferramentas que necessariamente pedem para que o programador insira determinados

tipos de **Elementos HTML** na página, ou a própria ferramenta vai fazer isso por você.

Por exemplo, vejamos o caso de uma galeria de fotos que está usando uma ferramenta chamada **Slick.js** para a criação de galerias de imagens. Para funcionar o **Slick** precisa usar os **Elementos HTML** configurados de um jeito específico, igual a documentação do **Slick** pede.

Esses **DOM's** eu chamo de **DOM do Pacote**, ou **Package DOM**.

CONTENT MANAGER DOM

O **Content Manager DOM** refere-se a plataformas que facilitam a criação, edição e gerenciamento de conteúdo, tal como o **WordPress**.

Quando você faz uma postagem na internet, dentro daquela caixinha escrita “**o que você está pensando?**”, o texto que você escreveu foi postado em uma caixa de texto chamada de **WYSIWYG** (literalmente “**O Que Você Vê É O Que Você Terá**”). Ao escolher a formatação do texto em itálico, por exemplo, esse painel vai inserir mais **Elementos HTML** na sua árvore **DOM**.

Toda vez o seu conteúdo requer um **Elemento HTML**, então será inserido mais elementos na sua página.

LAYOUT BUILDER DOM

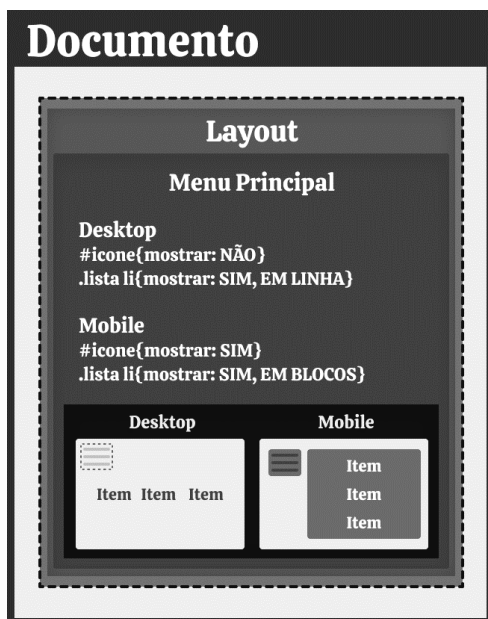
Não é só de conteúdo que vive o **DOM**, os elementos divisores podem servir como decoração e criação da arquitetura da página web.

Uma página web comumente possui cabeçalho, rodapé, parte principal e lateral. Ferramentas chamadas de **Layout Builder (Construtores de Layout)**, são recursos que ajudam na criação e organização do design do seu website.

MOBILE E DESKTOP DOM

O **Mobile DOM** é a representação do **DOM** em dispositivos móveis.

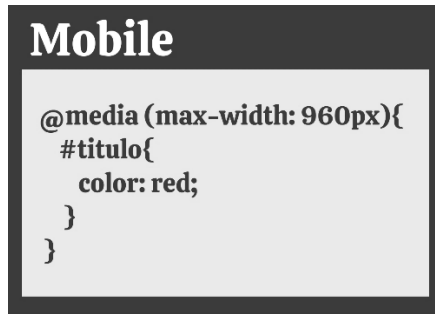
Em alguns casos, é comum usar alguns elementos específicos para cada tela, ou seja, tela do computador, do celular, do tablet e telas de retina. Exemplo fictício de código de estilo CSS:



O **Desktop DOM** é o **DOM** do computador de mesa (**Desktop**), mas nem todos o elemento do computador, estará perfeitamente alinhado em um tablet. Para isso é que serve o **DOM** do mobile, do desktop e outras visualizações necessárias podem precisar de mais do que apenas um **DOM**.

Utilizada como opção ou combinação perfeita, dentro do CSS você pode definir quais estilos o navegador apresentará em cada tela, são as "**media queries**".

As media queries fornecem uma alternativa para lidar com diversos tamanhos de telas, ex.:



No código CSS acima, somente telas que possuam **960 pixels de largura/width**, ou menores, poderão ver a cor vermelha no meu título.



É uma prática comum do mercado fazer páginas web que visualmente se enquadrem em celulares, tablets e computadores de mesa. Essa prática se chama **responsividade**.

DOM INJECTION

A **Injeção no DOM** é um processo que envolve adicionar novos elementos no **DOM** de uma página web. Geralmente é feito usando **JavaScript**.

Por exemplo, aquelas caixas chamadas “**pop-up**” que podem ser inseridas por meio do **JavaScript**.

A injeção de **DOM**, porém, pode ser usada por website maliciosos, para tentarem invadir o seu dispositivo e plantar um vírus.

ESTADO DO DOM

Os **Estados do DOM** referem-se a atributos condicionados à interação do usuário. Por exemplo: o usuário não está logado, mas existe uma mensagem no website para recepcionar os visitantes. A mensagem diz:

Seja bem-vindx, visitante.

Quando um visitante faz o login no website, o estado desse usuário passa de visitante para cliente, com isso o componente pode alterar o estado da variável “cliente” e com credenciais autenticadas mostrar o nome dele:

Seja bem-vindo, Antônio.

DOM WAR

É comum ver casos reais de websites de marcas globais, que tem uma verdadeira guerra de **DOM** ocorrendo nos bastidores.

Imagine esse cenário: Os programadores fazem seus elementos. Então o conteúdo das postagens, possuem outro **DOM**. O construtor do layout do site também vai injetar seu próprio **DOM**. Ferramentas como **Bootstrap** e **Tailwind** vão necessitar dos seus próprios **DOM's**. Se você quiser uma galeria que viu no **Codrops**, também terá injeção automática ou manual de **DOM**.

Quando tudo está no ar, o cliente pede uma alteração que nenhum desses sistemas faz e você tem que ser a pessoa que entende tudo o que está acontecendo para não fazer algo errado, mas são tantas camadas que você se perde.

Isso é muito comum. Então o programador vai mexer em um elemento, mas desalinha outros elementos e algumas coisas estranhamente não funcionam. Ele vai terminar o serviço aqui, mas algo quebra acolá. As alterações não surtem efeito, o programador experiente sabe que tem que fazer uma varredura e seguir traços de onde está vindo determinado código mal escrito. Cascatas e cascatas de ocorrências em desfavor da paz entre os **DOM's** e em desfavor da paz do programador.

É importante você ter a sua própria arquitetura de **Elementos HTML** e quanto menos ferramentas injetando elementos, mais fácil será o trabalho do programador **Front-End**.

Para agências web, o **DOM War** significa menor agilidade e pior aproveitamento de código. Pode ainda significar alta rotatividade de profissionais pela empresa. Ao longo de um ano inteiro, gerir projetos que não são bem organizados traz grandes perdas.

DESENVOLVIMENTO WEB MODERNO

Uma agência web deve ter a capacidade de iniciar um novo projeto de forma rápida e eficiente. Com um fluxo de desenvolvimento claro e eficaz.

Centralizados em uma biblioteca de códigos chamada de **Repositório Git**, eu preparei um molde com essas características, para você usar junto com a leitura deste volume. Você vai precisar de uma conta GitHub, acesse os e-books no material de apoio para aprender como criar essa conta.

O **molde** do livro se encontra no meu perfil **GitHub**, neste endereço de repositório:

www.github.com/miltonbolonha/next-boilerplate

GITHUB TEMPLATE

Um **GitHub Template** é um repositório de códigos contendo um **molde/template** para você iniciar de forma instantânea o desenvolvimento do seu próprio aplicativo (**PWA**) ou website.

Um **template** é um molde, é comum também ser chamado de **boilerplate**, ou seja, “chapa de metal”. O

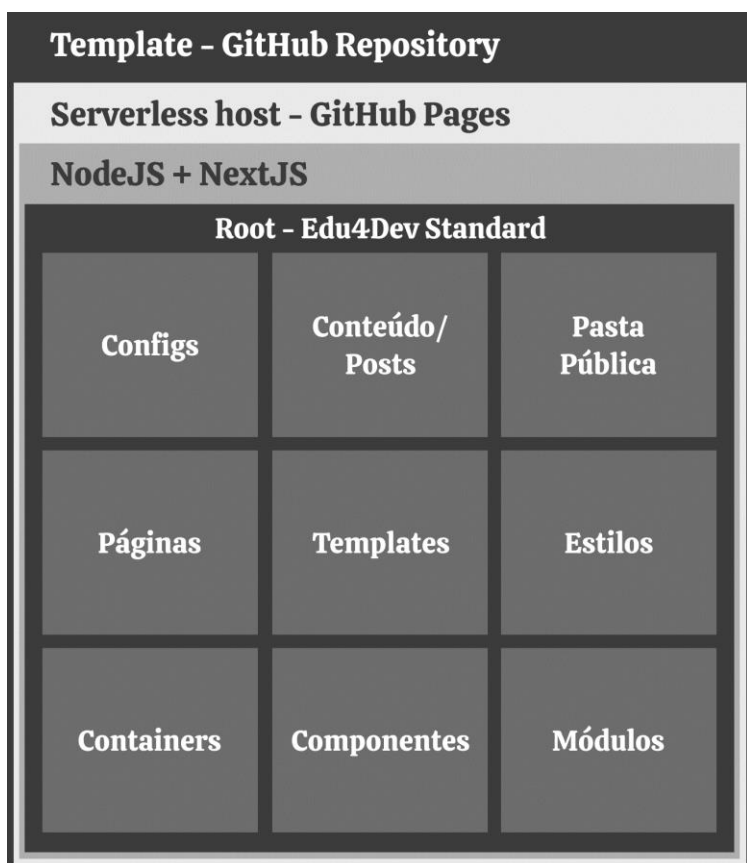
significado da palavra **boilerplate** se refere às chapas de metais que aquecidas têm a serventia de esquentar algo, por exemplo as chapas onde são feitos os lanches. A ideia é de que a chapa é o lugar para se cozinhar algo rápido e pré-pronto.

O nosso molde para aplicativos e websites é simples e compatível com diversos dispositivos; têm códigos em padrões internacionais de qualidade; e uma documentação que cobre dezenas de dicas e truques para o uso do mesmo.

Esperamos que use o nosso molde com a finalidade de explorar as suas próprias capacidades de programador. Use-o de forma comercial, pedagógica ou até mesmo filantrópica.

ESTRUTURA DO MOLDE

Primeiramente, o repositório de códigos está no **GitHub**. Esse repositório é especial, pois ele gera as páginas web, sem ter que contratar um serviço a mais para isso, é só usar o servidor (*serverless*) **GitHub Pages**.



O repositório contém os arquivos para se montar um website escrito em **NextJS**. Segue um resumo do padrão de organização **Next**:

- Configurações: Nome do negócio, telefone, redes sociais;
- Conteúdo e Posts
- Pasta Pública: Divulgue arquivos com o mundo na sua pasta pública;

- Páginas: Páginas web em branco, servem para serem preenchidas com o seu conteúdo;
- Templates: Moldes para páginas especiais;
- Estilos: Estilos visuais dos elementos (**CSS**);
- Containers: Recebe dados, altera e aplica filtros, depois repassa essas informações aos componentes visuais;
- Componentes: recebe dados e aloca-os nos elementos “**HTML/JSX**”;
- Módulos: pacotes de códigos estruturados para serem usados como ferramentas de programação.

AS SUAS PRIMEIRAS TECNOLOGIAS

Saia do “***não pare de aprender***” e entre para o “**Aprenda de Uma Vez Por Todas**”. Entender o básico nem sempre é uma tarefa fácil, porém eu me empenhei na árdua responsabilidade de escolher algumas poucas tecnologias para você ter um conhecimento sólido, que vai te acompanhar por toda a sua vida.

As seguintes escolhas foram baseadas em anos de estudo e experiência: Você vai ter o **Markdown** como padrão de escrita para os documentos oficiais dos seus produtos e serviços. O **JavaScript** vai te ajudar no **Front-**

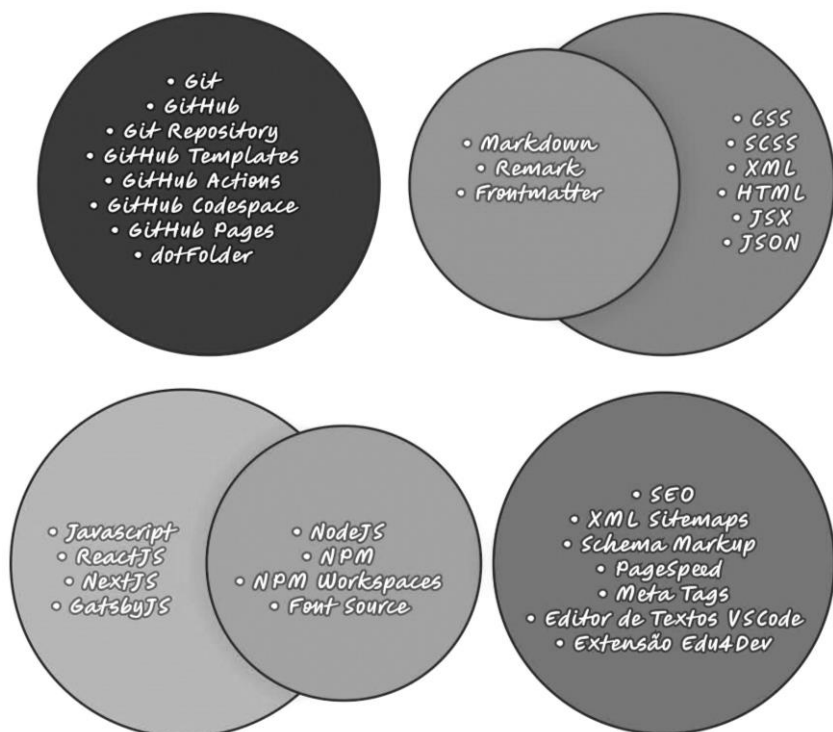
End a administrar blocos de informações e o **NextJS** vai te ajudar a construir as rotas do seu sistema, além disso o **next** será usado com capacidades de **Back-End**.

Além do **HTML**, que são esses blocos visuais com algum tipo de conteúdo, o **JSON** é uma forma de armazenar quantidades pequenas de informações, sem blocos visuais, sem nenhuma representação visual.

Por fim, o **CSS** irá pintar os blocos visuais, distorcer para caber dentro de determinado espaço na tela do cliente e assim compor toda a estilização do projeto.

ARQUITETURA DAS TECNOLOGIAS

Nesta área de programação, é bom você se acostumar a aprender novos nomes e muitas vezes que soam de forma esquisita. Veja nome das principais tecnologias do nosso projeto:



O SEU PRIMEIRO TEMPLATE

Eu preparei para você interagir com este capítulo, uma arquitetura de projeto real, que servirá como o seu primeiro molde.

É um **repositório GitHub** que serve como ponto de partida para as nossas atividades pedagógicas no campo da programação, com foco no desenvolvimento de aplicações web modernas e na hospedagem **serverless** do **GitHub Pages**.

Os nossos passos para experimentar essa tecnologia serão:

1. Clonar o repositório;
2. Configurar o arquivo `next.config.js`;
3. Enviar mudanças e comentar código;
4. Configurar GitHub Actions;
5. Iniciar esteira atualizando o repositório;
6. Acessar a sua página.

ANTES DE INICIAR

Primeiramente, você deve ter uma conta no GitHub. Faça o login com suas credenciais e busque pelo meu perfil:

miltonbolonha

Abaixo da minha imagem de perfil, clique em **“Follow/Seguir”**. Assim ficará mais fácil para você encontrar o repositório molde.

Na aba dos meus repositórios, você deve encontrar o repositório com o seguinte nome:

next-boilerplate

Ao encontrá-lo e visualizar o projeto, você notará no canto superior direito da sua tela um botão chamado **“Star/Estrela”**, clique nesse botão, assim você estará favoritando o repositório molde.

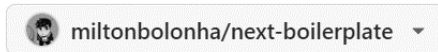
CLONAR O REPOSITÓRIO

Dentro do repositório, pressione o botão **“Use this template”**, no canto superior direito da tela e depois **“Create a new repository”**:



Em seguida, dê o nome ao seu repositório de “**next-boilerplate**”. Você pode escolher outro nome de repositório, mas terá que fazer algumas configurações adicionais.

Repository template

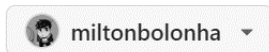


Start your repository with a template repository's contents.

☐ Include all branches

Copy all branches from miltonbolonha/next-boilerplate and not just the default branch.

Owner *



Repository name *



✔ meu-primeiro-molde is available.

Dê início a criação do repositório clicando em “**Create repository**”:



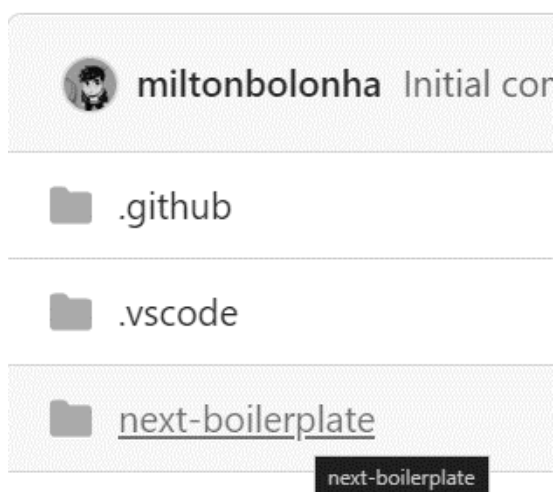
O seu repositório foi criado, para quem mudou o nome do repositório para algo customizado, temos um

passo a mais, vamos configurar as coisas para que a sua página web comece a ser gerada.

CONFIGURANDO O NEXT.JS

Caso você mudar o nome do seu repositório, o próximo passo é exclusivo para você. No exemplo, vamos usar o nome de repositório: “**meu-primeiro-molde**”.

Acesse a pasta “**next-boilerplate**”:



Abra o arquivo “**next-config.js**”:



Encontre o botão de editar, que é um lápis:



Edite a linha 3 do arquivo **next.config.js**:

Edit
Preview

Spaces
2

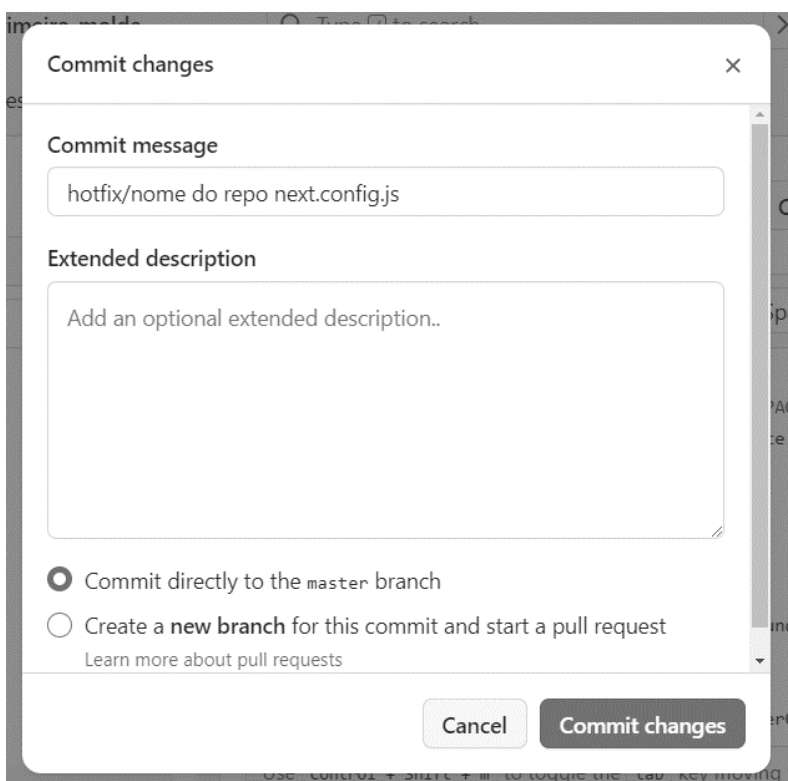
```

1  /** @type {import('next').NextConfig} */
2  const isGithubPages = process.env.GITHUB_PAGES || false;
3  const repo = "meu-primeiro-molde"; // edite aqui
4
5  const nextConfig = {
6    reactStrictMode: true,
7    output: "export",
8    trailingSlash: true,
9    basePath: isGithubPages ? "/" + repo : undefined,
10   images: {
11     loader: "custom",
12     loaderFile: "./src/containers/imgLoaderContainer.js",
13   },
14   env: {
15     IS_GITHUB_PAGE: isGithubPages,
16     THEME_FOLDER: repo,
17   },
18 };
19
20 module.exports = nextConfig;

```

Confirme as mudanças clicando no botão “**Commit changes**” e faça um comentário sobre essa alteração:

Commit changes...



The screenshot shows a 'Commit changes' dialog box with a close button (X) in the top right corner. It contains a 'Commit message' text field with the text 'hotfix/nome do repo next.config.js'. Below this is an 'Extended description' text area with the placeholder text 'Add an optional extended description..'. At the bottom, there are two radio button options: 'Commit directly to the master branch' (which is selected) and 'Create a new branch for this commit and start a pull request'. A link 'Learn more about pull requests' is located below the second option. At the very bottom of the dialog are two buttons: 'Cancel' and 'Commit changes'.

Commit changes

Commit message

hotfix/nome do repo next.config.js

Extended description

Add an optional extended description..

☒ Commit directly to the master branch

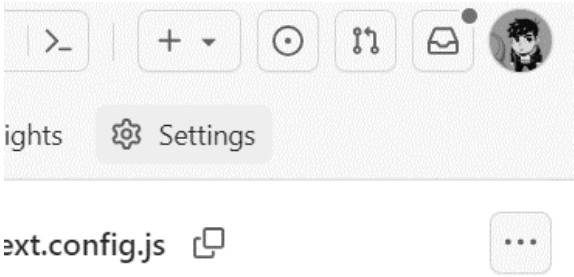
☐ Create a new branch for this commit and start a pull request

[Learn more about pull requests](#)

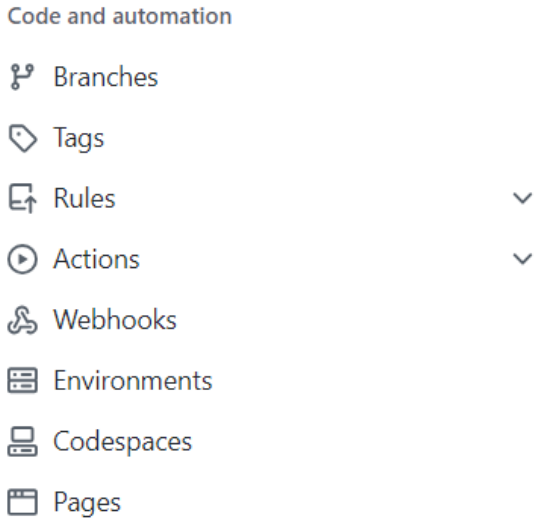
Cancel Commit changes

CONFIGURANDO O GITHUB PAGES

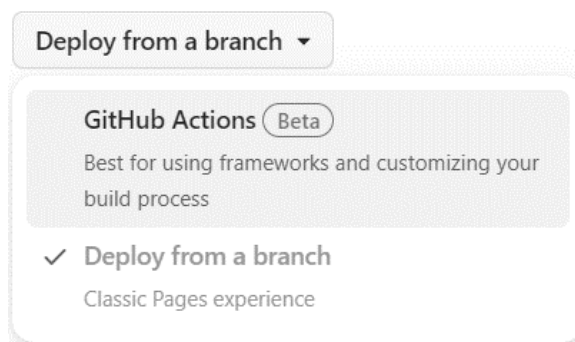
Após isso, acesse a aba **Settings** do seu repositório:



Na barra lateral esquerda, acesse “**Pages**”:



Você vai configurar o servidor **GitHub Pages** escolhendo a opção de montar o seu website, por meio das **GitHub Actions**:



Mude o deploy para GitHub Actions

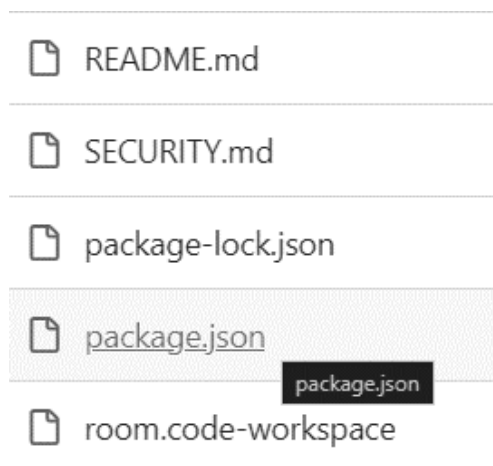
INICIANDO A ESTEIRA DE EVENTOS E MONTAGEM

A sua esteira de montagem de aplicativos e websites está pronta para ser iniciada. Esse serviço está configurado na pasta das **GitHub Actions** e está esperando por qualquer mudança para iniciar uma esteira de montagem.

Para iniciar a esteira automática precisamos de uma mudança qualquer. Volte a aba de código do seu projeto acessando a aba “**Code**”:



Encontre na tela inicial do seu repositório, um arquivo com o nome “**package.json**”.



Esse é um arquivo central de um repositório contendo tecnologia **NodeJS**. Nele vamos alterar apenas a numeração da versão atual do nosso website:

meu-primeiro-molde / package.json



miltonbolonha Update package.json 1285b73 · now

History

42 lines (42 loc) · 1.06 KB

Code

Blame

Raw



```
1  {
2    "name": "next-boilerplate-workspace",
3    "version": "1.0.0",
4    "workspaces": [
5      "next-boilerplate"
6    ],
```

Edite a linha 3 (**package.json:3**), o número que está em “**version**” se refere a versão do seu website **1.0.0**. Mude a versão para **1.1.0**.

Agora que você fez a primeira atualização, a esteira de eventos de construção do seu website foi iniciada.

ACESSANDO A SUA PÁGINA

Volte a tela inicial do código do seu repositório, também chamada de raiz do repositório. Na lateral direita, clique no ícone pequeno em formato de engrenagem ao lado de “**About**”:

About



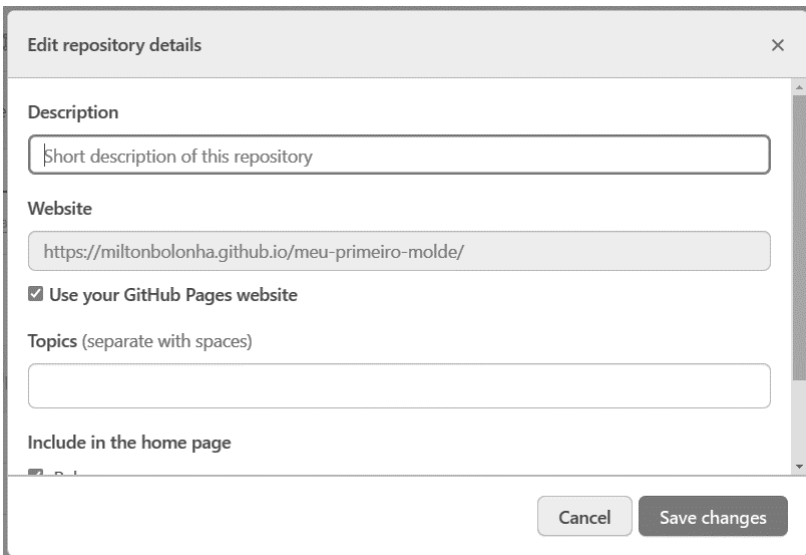
No description, website, or topics provided.

 Readme

 MIT license

 Code of conduct

No *pop-up* que aparecer, selecione na seção Website o campo **“Use your GitHub Pages website/Usar meu website do GitHub Page”** e então salve sua mudança acionando **“Save Changes”**:



Edit repository details

Description

Short description of this repository

Website

<https://miltonbolonha.github.io/meu-primeiro-molde/>

☒ Use your GitHub Pages website

Topics (separate with spaces)

Include in the home page

Cancel Save changes

Agora, a barra da lateral direita do seu repositório tem um link para o website que você acaba de criar:

About

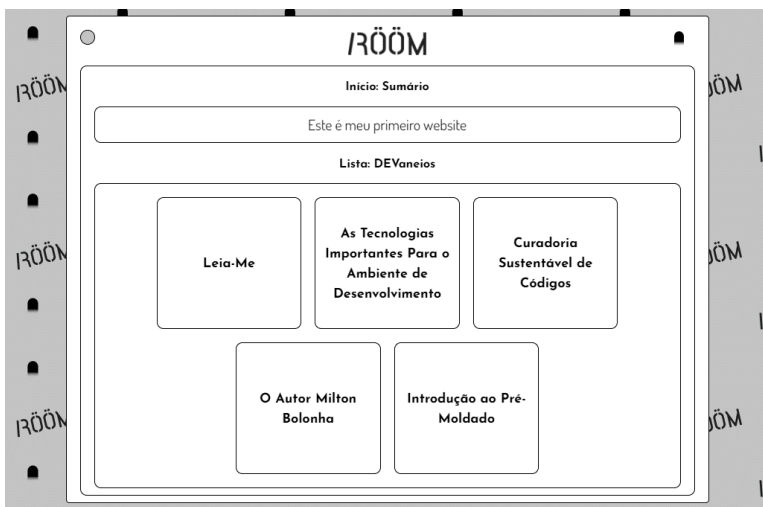


 miltonbolonha.github.io/meu-primeiro-...

 Readme

 MIT license

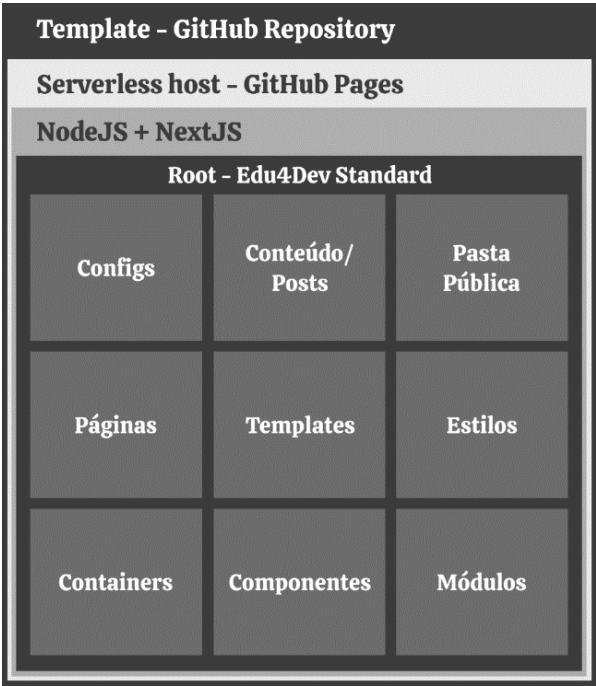
Clique no endereço e aceda ao seu primeiro website moderno:



Se você está vendo uma tela parecida com a imagem anterior, parabéns, você conseguiu usar o seu primeiro **Molde GitHub**.

ARQUITETURA DO MOLDE

Vamos conhecer por dentro desse sistema de inicialização rápida chamado **molde/templates**.



O SERVIDOR GITHUB PAGES

O seu sistema estará disponível na internet em um servidor moderno e grátis, chamado de **GitHub Pages**.

Um servidor é o computador que guarda os arquivos do seu website e o disponibiliza ao mundo.

As configurações do **GitHub Pages** estão no painel administrativo do seu repositório, na aba “**Settings**”. Como vimos anteriormente, selecionando o submenu **Pages**, você poderá configurar as **GitHub Actions**. Essas **Actions** estão na pasta ``./github/workflows``.

É um grande benefício usar servidores modernos e terceirizados, deixando a segurança garantida pela **Microsoft GitHub**, com alta performance em **SEO** e **PageSpeed**.

Você pode usar o **GitHub Pages** para quantidade ilimitada de projetos. Você pode usar o **GitHub Pages** para hospedar um website sobre você, sua organização ou seu projeto diretamente de um repositório no **GitHub**.

O servidor **GitHub Pages** não é um servidor igual aos outros, ele somente armazena arquivos estáticos, por isso é chamado de **Serverless** (pouco ou sem servidor).

O **serverless** não possui algumas funções especiais que rodam no lado do servidor, isso quer dizer que você tem tudo o que precisa para um website ou app rápido e moderno, mas muito simples e sem funcionalidades super avançadas.

Para um website profissional eu recomendo usar o **Netlify** ou o **Vercel**. O propósito de usarmos **GitHub Pages** é pedagógico, mas já garante a mesma

experiência de ferramentas completas como o **Netlify** e o **Vercel**.

GITHUB ACTIONS E SERVERLESS

As **GitHub Actions** são as engrenagens de automação (similar ao **Docker**) da sua esteira de eventos e montagem. São responsáveis pelas primeiras etapas na esteira de montagem do seu app.

Dentro da pasta “**.github/workflows**” você encontrará sendo declarado esses ambientes em arquivos ***.YML**:

- Construção do molde **NextJS** e upload para o **GitHub Pages**: **nextjs-github-pages.yml**;
- Empacotamento e disponibilização de versão nova: **pre-releases.yml**.

Ao copiar o molde e configurá-lo, são esses arquivos que darão início a esteira de construção dele. Todas ações programadas podem ser vistas em execução na aba “**Actions**”.

Um dos passos da construção do molde é o upload dos arquivos para o servidor (**serverless**) **GitHub Pages**. Nesse passo da nossa automação, o sistema vai fazer o upload dos arquivos gerados do seu website para o sistema da **GitHub Pages** ou qualquer outro que você configurar.

USANDO O REACTJS DA META/FACEBOOK

O **React.js** é uma biblioteca **JavaScript** para construção de interfaces de usuário.

O **React** permite construir interfaces de usuário a partir de peças individuais chamadas **componentes**. Por isso, podemos dizer que ele é orientado a componentes.

Crie apenas uma vez os seus componentes **React** como **<GaleriaDeFotos />**, **<BotaoDeLike />** e **<ListaDePosts />**. Em seguida, reutilize-os em telas inteiras, páginas e entre projetos.

O **React** foi projetado para permitir que uma agência web orchestre componentes entendidos e escritos por pessoas comuns, assim trabalhamos com harmonia junto a toda equipe na hora do desenvolvimento de um projeto.

Literalmente, está todo mundo usando o **React**! Dois milhões de pessoas programadoras do mundo todo visitam a documentação do **React** todos os meses.

O **React** é a escolha certa, junto a ele a tecnologia que mais cresceu e tomou conta do mercado nesses últimos anos foi o **Next.js**, que é feito com **React.js**.

JSX

O pessoal da **Meta**, ao fazer o **React** deixou os elementos visuais por conta de uma ferramenta já existente, o **JSX**, que lembra muito os **Elementos HTML**.

O **JSX** é tipo um **Super Elemento HTML**, ou como eu gosto de chamar é o “**AtomickBlock/BlocoAtômico**”.

É também uma extensão do **JavaScript** popularizada pelo **React**.

A melhor forma de usar esses blocos é separando a parte visual da lógica. Colocando no topo do seu componente, um container de códigos com as regras de negócio (*ver adiante*).

COMPONENTES (BLOCOS VISUAIS)

Os **Componentes** são os elementos para nós construirmos o nosso projeto de forma visual, porém são blocos turbinados com muitas funções.

Por exemplo, ao fazer um componente de botão de like, podemos invocá-lo em qualquer lugar da seguinte maneira:

```
<BotaoDeAcao />
```

Digamos que dentro do componente, foi programado que esse botão teria o like e outras reações. Esse botão poderia ser um “amei” e para escolher entre um ou outro, temos o seguinte componente:

```
<BotaoDeAcao tipo="curtir" />
```

```
<BotaoDeAcao tipo="amei" />
```

Explicando o código anterior, você lê o componente **BotaoDeAcao** e um **Atributo** chamado “tipo”. Componentes sempre começam com a letra maiúscula e seguem o padrão de nomenclatura chamado **PascalCase**.

CONTAINERS (LÓGICA)

Nem sempre os dados armazenados, são aqueles que vão para o usuário final. Interações avançadas com o usuário também modificam a navegação. Os **Containers** são onde preparamos os dados para serem passados para os **Componentes**, para só enfim serem apresentados para o cliente.

Por exemplo, a data de uma postagem de *blog* é armazenada em formatos para computador e não em formato humano. Onde vemos “**31 de janeiro de 2023**”, essa data no nosso molde estará dessa maneira:

2023-01-31T09:30:00+00:00

Precisamos usar ferramentas do **JavaScript** para manipular essas informações.

OS DADOS

Como vimos anteriormente, os **Containers** recebem os **Dados** para serem filtrados e repassados para os componentes. Alguns desses dados são:

- Dados do negócio
- Termos Gerais
- Postagens
- Traduções

NEXT.JS

O **Next.js** é um manipulador de rotas e páginas web. Ele está dentro da pasta **next-boilerplate**, sendo declarado no arquivo **package.json**. É uma forma de criar rotas na web e apresentar páginas com conteúdo simples, especiais, privados, criptografados e *streaming*.

Todo o código do **Next.js** está aberto (**open source**) no **GitHub** e é um dos repositórios mais populares do mundo.

Se popularizou por ser feito em **React.js** e por ser uma maneira fácil de criar um aplicativo ou website moderno em segundos.

Marcas mundiais usam o **Next**: Nike, United Air Lines, Hilton, Red Bull, Hyundai, Marvel, Ferrari, Sir Elton John, etc.

Essas páginas podem acessar rotas externas de serviços terceirizados, por exemplo, para acessar os dados de um cliente, em um banco de dados terceirizado.

NEXT.CONFIG.JS

Este é o arquivo de configurações globais iniciais do **NextJS**:

next-boilerplate/next-config.js

Quando o seu repositório template foi criado, você optou por usar um nome padrão do repositório, ex.: “**next-boilerplate**”.

Caso você tenha alterado o nome do seu repositório, você deve atualizar este arquivo alterando o nome antigo do repositório, para o novo nome.

No caso de você ter um domínio (**www**) personalizado, você deve atualizá-lo também.

VERCEL & NETLIFY (SERVERLESS)

Para podermos criar um sistema sem limitações de servidor, nem limitações comerciais, você pode querer contratar um plano de hospedagem moderna tanto da **Vercel**, quanto da **Netlify**. Ambos servem como hospedagem **serverless**.

A **Vercel** é a empresa que está por trás da tecnologia **NextJS**. Já a **Netlify** adquiriu o **GatsbyJS** no começo de 2023.

Eu recomendo altamente usar um desses dois servidores.

NODE E NPM PACKAGES (PACKAGE.JSON)

O **Next** está na pasta **next-boilerplate**, mas eu quero que você perceba, que na raiz desse repositório existe um arquivo chamado “**package.json**”.

Esse arquivo é o responsável por orquestrar todo o ambiente de trabalho do nosso repositório.

Lembra-se da criação do universo? Se o seu aplicativo fosse um universo, está nesse arquivo o início de tudo, com o roteiro principal da construção dele.

O “**package.json**” armazena informações os nomes e versões de todos os pacotes instalados, para serem executados no ambiente **Node**. É também onde você configura os comandos de construção do seu sistema.

NPM WORKSPACE (MONOREPO)

Ter mais de um “**package.json**” por projeto **Node** não é o comum. Essa é uma técnica avançada do **Node.js**, onde um repositório possui diversos projetos (**Monorepo**) e são orquestrados na esteira de eventos pelo **NPM Workspace**.

É muito comum você precisar de mais de um repositório para fazer **Back-End** e **Front-End**. Juntando tudo em um repositório e usando o **NPM Workspace**, o seu sistema será mais leve e inteligente. Aumenta significativamente o entendimento e leitura do projeto.

Usando **NPM Workspace**, os arquivos e pastas estarão muito mais organizados e poderão ser reaproveitados entre si.

É um ótimo padrão para criar temas **Gatsby** e **Gatsby Plugins**.

Eu prefiro usar apenas o **NPM**, em detrimento da camada adicional chamada **Yarn**, igualmente útil.

Existem várias ferramentas para a técnica de controle de **Monorepo**: **Yarn**, **Turborepo**, **Lerna**, **Bit**, etc.

Se você quer ser um profissional valioso no mercado, com maiores oportunidades, aprenda todas as técnicas e mantenha um repositório por cada uma delas, como prova do seu conhecimento.

CONFIGURAÇÕES BÁSICAS E CONTEÚDO

As principais tecnologias que você utilizará para inserir as informações mais básicas no seu website são as seguintes: **JSON** e **Markdown**.

CONFIGURAÇÕES BÁSICAS

Dentro da pasta **next-boilerplate/src/configs** você encontrará dois arquivos **JSON's** para configurar o seu website:

- main-infos.json
- main-menu.json

main-infos.json

É neste arquivo que vamos alterar o nome do negócio, telefone, redes sociais, conteúdo das páginas estáticas.

main-menu.json

Este é o arquivo responsável pelos nomes e links do menu principal, mas atenção, alterar o menu não altera o nome de nenhuma página.

CONTEÚDO/DOCUMENTOS

Dentro da pasta “**next-boilerplate/content**” estarão pastas contendo os arquivos **Markdown** com o conteúdo do website.

A nomenclatura das pastas deve servir para facilitar o entendimento, são mais comuns as pastas **posts/postagens** e **pages/páginas**.

Você pode querer colocar uma pasta para cada **tipo** de conteúdo, por exemplo: filmes, produtos, etc. Tudo o que pode virar um documento catalogável, estilo arquivo.

PASTA PÚBLICA

Você encontrará dentro do molde uma pasta para colocar arquivos públicos, tal como imagens, pdf's, pastas inteiras, etc.

`next-boilerplate/public`

Todos os arquivos dentro dessa pasta estarão disponível no seu endereço de website, em sua raiz. Por exemplo, o cliente quer expor o catalogo dele que está em pdf, você copia para essa pasta e então o sistema irá expor o arquivo em seu domínio, dessa forma:

`www.dominio.com.br/catalogo.pdf`

CRIAÇÃO DE PÁGINAS E ESTILOS

Como dito no começo, a internet é feita de páginas web. Na hora de criar no **Front-End** essas páginas, você usará como tecnologias principais: **React**, **JSX**, **ECMAScript**, **JavaScript**, **CSS** e **HTML**.

PÁGINAS EM JS

Criar páginas em **JavaScript** é muito fácil, crie arquivos **JS** com o nome da sua página a partir da seguinte pasta:

next-boilerplate/src/pages

Essas páginas estão escritas em **JavaScript** usando as **Arrow Functions**.



Cada arquivo da pasta de páginas, gera uma nova página para os nossos visitantes (com exceções).

Algumas páginas básicas são:

- **index.js** (*nome-da-pagina.javascript*).

Lembrando que diversas páginas formam um livro, index significa sumário e funciona como um atalho para as sessões do seu web site. É também a página inicial do seu sistema.

- **404.js**

Assim como em um semáforo, os sistemas web possuem sinais para avançar, parar e prestar atenção.

Na web, esses sinais são respostas para a tentativa de pegar uma rota, então o servidor retorna o estado da página em um número (**HTTP Status Codes**) **200, 300, 400**. O número **404** representa uma rota errada, sem uma página para exibir. É necessário adicionar uma página para a ocorrência chamada de **Erro 404**.

- **contato.js**

O tipo mais comum de página de contato é aquela com um formulário para enviar uma mensagem para os donos do website. O estilo “**LinkTree**” está sendo usado no nosso molde.

- **sobre.js**

Praticamente todo sistema vai querer exibir uma página falando sobre o próprio negócio. Essa é a página “**Sobre Nós**”;

- **[slug].js**, esse arquivo é o responsável por orquestrar a transformação do **Markdown** em páginas **HTML** e rotas de navegação.

PÁGINAS TEMPLATES

As páginas de molde/templates são usadas para apresentar o conteúdo **Markdown** de uma forma específica customizada. São páginas em **JS** também.

ESTILOS DOS BLOCOS

Todo o conteúdo do website estará dentro de blocos e o **CSS** irá definir os estilos visuais gráficos para esses blocos. Eles dão o visual e a personalidade. Definimos cores, formas e padrões para criar uma experiência visual incrível.

Esses arquivos estão escritos no estilo **SCSS**, que é uma técnica do **CSS** e fica na pasta:

next-boilerplate/src/styles.scss

Nesse arquivo você encontrará os seguintes atalhos para os demais arquivos **SCSS**:

- Definições Globais e Variáveis
- Páginas
- Componentes
- Módulos
- Outros

A “CENTRAL” DO SEU FRONT-END

Vamos dar uma olhada agora em três arquivos que estão dando suporte ao nosso layout dentro do **Next.js**.

`_app.js`

Next.js usa o componente **App** para inicializar as páginas web. É aqui onde o programador pode importar ferramentas, fontes, criar um layout compartilhado entre páginas, injetar dados adicionais nas páginas, adicionar estilos globais **SCSS**.

Também usamos técnicas avançadas de compartilhamento de informações dentro do aplicativo, como o **React Context**.

`_document.js`

Neste arquivo `_document.js` podemos definir idioma, carregar scripts antes da interatividade da página, inserir **`Analytics`**, dentre outras funções menos visuais.

`[slug].js`

Esse é o arquivo responsável por garantir o conteúdo das rotas geradas por cada arquivo **`MarkDown`**.

ANÁLISE DE WEBSITES E SEO

Nesta seção vou te apresentar alguns conceitos que identificam um bom website.

É muito importante para a saúde de um projeto web ter um bom trabalho de implementação de **SEO**, ou seja, ser encontrado nas buscas web e disputar muito bem contra a concorrência.

O **SEO** (otimização para buscas de internet) é uma técnica para escrever informações do seu negócio de forma a melhorar a visibilidade do seu site nos resultados das pesquisas na internet.

A otimização de **SEO** envolve técnicas no website e no conteúdo para aumentar as visitas orgânicas do seu funil de vendas. Técnicas como a escolha de palavras-chave relevantes, criação de conteúdo de qualidade e melhorias na estrutura do site.

Se difere dos anúncios (**ADS**) da internet, pois o **SEO** considera a qualidade do código do seu website e do seu conteúdo para mostrar resultados para o cliente das buscas on-line. Já os anúncios vão aparecer forçosamente, por causa de pagamento e sem concorrência na qualidade.

Todos os envolvidos em um projeto web devem conhecer, dar prioridade e discutir esse tópico.

MAPAS DO SITE XML

É importante você dizer aos motores de busca quais são todas as páginas web que você possui, para isso existem os mapas do site.

Você pode querer separar páginas comuns de páginas ultra otimizadas (**Amp – Páginas Aceleradas para Mobile**) e criar diversos mapas, contendo os links do seu website.

Isso facilita que os motores de busca (ex.: **Bing**) encontrem e indexem seu conteúdo com mais eficiência, todas as páginas web criadas.

MAPA DE DADOS DO NEGÓCIO (SCHEMA MARKUP)

Depois de fazer uma nova página web, ou uma linda galeria de fotos no seu website, contendo o seu trabalho, é importante dizer ao universo de dados que aquela página se refere ao seu portfólio e que cada item da galeria reflete um trabalho específico.

Não basta que o seu negócio esteja escrito ao usuário bem explicadinho, mas também é necessário que você explique em linguagem especial para os robôs, que circulam a internet visitando websites e coletando dados.

Cada um de seus dados, é como uma informação dentro de um mapa. Estamos falando do mapa de dados do negócio. Informações como horário de funcionamento, endereço, nome do produto, telefone, logotipo, etc.

É por meio de dados estruturados que o seu website entrará no **Metaverso**, ou não. Esses dados, quando estruturados, são chamados de **Metadados** e seu coletivo gera o tal **Metaverso**.

Esses **Esquemas** vão gerar **Cards**, os cards são códigos adicionados às páginas web, eles ajudam os mecanismos de busca a entender o seu conteúdo.

Para saber se um website possui esses cards gerados do **Schema**, você pode usar a seguinte ferramenta de análise e validação de **Schemas**:

validator.schema.org

Quando um robô de buscas encontra um website com dados estruturados, esse website ganha maior relevância dentro do universo de dados, portanto ele fica com maior destaque nos resultados das pesquisas de internet.

VELOCIDADE DE CARREGAMENTOS (PAGESPEED)

A velocidade de carregamentos do seu site, é um fator crucial para a experiência do usuário e para o ranking nos resultados de busca.

Páginas web lentas e que demoram para serem iniciadas, tem alto índice de rejeição pelos usuários. Basicamente, o ser humano não espera a página carregar, tem que ser tudo instantâneo.

O **PageSpeed** refere-se à otimização do tempo de carregamento das páginas, por meio de técnicas como compressão de imagens, redução de códigos desnecessários e uso de **cache**.

As métricas avaliadas também levam em consideração otimizações de código e erros de padrões.

Você deve sempre utilizar essa ferramenta para saber se o código de um website está escrito de forma profissional. Essas ferramentas são bem visuais e te mostrarão em escalas simples se o website é bom ou não.

Algumas dessas ferramentas são:

Google PageSpeed, Pingdom, GTMetrix, dentre outras.

META TAGS

As **Meta Tags** são informações do negócio, ou da página, inseridas por meio do código **HTML**, para serem lidas NÃO por humanos e sim por esses sites de busca, grosso modo.

Informações como título, descrição, palavras-chaves, idioma e mesmo os **Schemas Markup** serão inseridos por aqui.

Uma outra **tag** que ajuda muito no desenvolvimento do seu website, são as **Meta Tags** que geram métricas de uso dos usuários do seu site, são também chamadas de **Analytics**.

DNS SERVER

Todos os servidores, os computadores que armazenam o seu código, eles possuem um nome e alguns outros dados para serem identificados.

O **Domain Name Server**, ou apenas **DNS**, é exatamente isso. Quando você estiver mais experiente, é por lá que você vai orquestrar o endereço do seu website.

O domínio básico, que todos conhecemos, é o famoso **www**. Existem outros domínios que são apenas numerais, chamados de **IP**. Ainda podemos criar

máscaras e **redirecionamentos**, ou mesmo **subdomínios**, tal como:

admin.meudominio.com.br

CROSS BROWSER

Cross Browser significa “**Em Todos Os Navegadores Web**” e refere-se à compatibilidade de um website com diferentes navegadores.

É fundamental que os desenvolvedores testem os seus projetos em vários navegadores para garantir que a experiência do usuário seja consistente e funcione corretamente em todas as plataformas.

TÉCNICAS AVANÇADAS

Acima de tudo, um projeto web contará com pessoas. Às vezes, essas pessoas precisarão de um painel administrativo ou uma outra técnica avançada vinda do mundo da programação

A seguir algumas das técnicas avançadas de um projeto web.

PAINEL ADMINISTRATIVO

O painel administrativo é uma ferramenta fundamental que permite aos usuários gerenciar e atualizar facilmente o conteúdo do website ou aplicativo.

As vezes chamado de **Gestor de Conteúdo (CMS)**. **NetlifyCMS** e **WordPress** são algumas plataformas modernas que possibilitam essa funcionalidade.

CMS, DMS E MARKDOWN

CMS significa **Gestor de Conteúdo**. É a parte administrativa de um website.

Em vez de usar um **CMS**, talvez você queira experimentar usar um **DMS** (sistema de gerenciamento de documentos).

A técnica chamada de **Gestão de Documentos**, utiliza o padrão de escrita **Markdown** para arquivar os documentos do seu negócio em uma pasta geralmente nomeada de “**content/conteúdo**”, sem a necessidade de uma estrutura grande, como no caso do **CMS**.

Qual é a diferença entre uma página da web administrada por **CMS** e uma administrada por um simples documento **Markdown**?

Nessa técnica não se usa nem um banco de dados, nem um painel administrativo para gerir os documentos, eles são criados como arquivos, diretamente no repositório **Git**.

Para uma postagem de um **blog Wordpress** são necessários um banco de dados e um servidor, no mínimo um profissional experiente para administrar isso tudo.

Os clientes da programação querem websites que não lhe rendem muitos encargos.

O custo de se manter um servidor pode ser caro e o retorno com essa arquitetura pode encarecer o seu projeto.

O principal fator para se usar um **DMS** é que manter um **DMS** não tem custo.

HEADLESS CMS

Existe uma técnica de gestão de conteúdo em que se usa um serviço web para administrar esse conteúdo, de forma que ele não está atrelado ao seu sistema visual.

O seu sistema conecta e lê o conteúdo que está em uma outra plataforma, então coloca os dados no **Front-End** programado. É chamado de **HeadlessCMS**, ou gestor de conteúdo sem **Front-end**.

Algumas plataformas de **HeadlessCMS** são: **WordPress, Strapi, Tina, Contentful, Jekyll Admin**, dentre outros.

Atualização dinâmica dos produtos, preços e descrições, são alguns dos dados armazenados por sistemas **HeadlessCMS** e a parte gráfica fica toda com o **React (Next ou Gatsby)**.

NETLIFYCMS + GITHUB OAUTH

O **NetlifyCMS** oferece uma interface de usuário intuitiva e fácil de usar para gerenciar conteúdo, ele é um sistema **HeadlessCMS**.

Ao integrá-lo com a autenticação do **GitHub (GitHub oAuth)**, os colaboradores autorizados podem fazer login no painel administrativo usando suas credenciais do

GitHub, proporcionando segurança, controle de acesso ao conteúdo e sem custos.

WORDPRESS + GATSBYJS OU NEXTJS

A combinação do **WordPress** (WP) sendo usado como **HeadlessCMS** e as ferramentas **GatsbyJS** ou **NextJS** como **Front-End** é uma ótima abordagem.

Serve muito bem para a criação de websites institucionais, em sua maioria das vezes estáticos, que usa os recursos robustos do **WordPress** como **Back-End**.

Exemplo de uso, um site de uma editora que precisa de um sistema robusto para gerenciar e exibir seu catálogo de livros. O **WordPress** é utilizado para gerenciar os detalhes dos livros, autores, capas, etc. Enquanto o **GatsbyJS** transforma esses dados e exibe uma página para o seu cliente no seu website, criando uma página de produto para cada livro cadastrado.

VANILLAJS, ECMAScript E ARROW FUNCTIONS

O **Javascript** mesmo é só um, mas possui muitas camadas que devem servir para facilitar na hora de escrever os códigos.

É de bom tom, estiloso e de alta qualidade, escrever Javascript puro, dentro da programação que não pede uma camada a mais. Nós programadores chamamos o código **Javascript** puro de **VanillaJS**.

Outra técnica de escrita **Javascript** se chama **ECMAScript**. Neste padrão, podemos escrever pedaços do nosso código de formas mais intuitivas, por meio das “**Arrow Functions/Funções Usando Seta**”.

Com as **Arrow Functions** o código fica mais legível e fácil de compartilhar com a equipe e novos membros.

Nota: O **ECMAScript** precisa de uma ferramenta chamada compilador **Babel**.

MARKDOWN AVANÇADO

O **Markdown** é o padrão de escrita de documentos, como vimos anteriormente. É útil para guardar informações como data de criação do documento, autor do documento.

Os documentos .MD podem ter cabeçalhos com conteúdos especiais.

chave: Valor

Conteúdo de postagem comum

São atributos com chave e valor, com detalhes adicionais do documento. O cabeçalho se inicia e termina com três traços “---”. Dentro dessa marcação você vai anunciar o nome de um identificador (**chave**) e o seu conteúdo (**valor**), como no exemplo abaixo:

título: Curadoria Sustentável de Códigos

data: 2023-08-16 00:00:02

imagem: "homem-computador.jpg"

descrição: "Algumas dicas."

categoria:

- programação

IMAGENS RESPONSIVAS

O **GatsbyJS** se destaca pelo seu plugin: **Gatsby Image Plugin**, que serve para adicionar imagens adaptáveis a tela (**responsivas**) ao seu website.

Técnicas para imagens responsivas ajudam na hora de carregar o seu website. O **Gatsby Image Plugin** cuida da parte difícil de produzir imagens em vários tamanhos e formatos e entregam a imagem certa para a tela certa. Outros módulos com as mesmas capacidades também podem ser encontrados via **NPM**. Destaca-se o **Sharp**, que tem aproximadamente 2 milhões de downloads semanais.

WebP é um formato de arquivo de imagem do **Google** que oferece melhor compactação do que **JPEG** ou **PNG**. É o melhor padrão para servir imagens na internet.

FONTE DE FONTES

Font Source é uma coleção de **fontes** de **código aberto** empacotadas em pacotes **NPM** individuais para **auto hospedagem** de fontes. Isso é importante principalmente para a lei europeia de proteção de dados.

Algumas considerações para usar:

- Auto hospedagem de fontes evita problemas com leis regionais, como a lei europeia;
- Suas fontes carregam offline;
- Privacidade do Google, uma vez que o Google usa todas as ferramentas possíveis para vasculhar o seu website e/ou app. Muitas vezes a privacidade é mais importante.

PWA - PROGRESSIVE WEB APP

PWA é uma técnica avançada para fazer aplicativos de maneira simples e ter o poder de acessar certos componentes do dispositivo que usar o app desenvolvido.

O PWA pode:

- Enviar notificações *push*;
- Funcionar *offline*;
- Inserir ícone na tela principal do celular;
- Acesso à câmera, contato, arquivos e geolocalização.

O **PWA** possui menor capacidade do que os chamados códigos nativos dos dispositivos e não é tão veloz, por isso não é escolhido como plataforma para apps grandes, como um jogo.

O **React Native** é usado para programar tanto em **Android** quanto **IOS**. Em vez de usá-lo, uso o **PWA** e o **ReactJS**.

GIT DEPENDABOT

Git Dependabot é um recurso oferecido pela **GitHub** que ajuda a manter o seu projeto atualizado automaticamente. Ele monitora regularmente as dependências do seu código, como bibliotecas, frameworks e pacotes, em busca de novas versões disponíveis.

Quando uma atualização é identificada, o **Dependabot** gera automaticamente uma solicitação para atualizar essas dependências em seu repositório.

A verificação de código analisa o seu código em busca de problemas de segurança também. Se o programador expor uma chave-segura, ou um token, o **Dependabot** vai avisar.

COMENTANDO CADA ATUALIZAÇÃO GIT

Quando você atualizar qualquer coisa no **Git**, ele vai te pedir que você faça um comentário sobre as mudanças realizadas.

É responsabilidade do programador realizar pequenos comentários a cada uma dessas atualizações, com detalhes do trabalho realizado e como as partes afetadas foram mudadas.

Feature (novidade)

Comece o seu comentário **Git** com a palavra "**feature**" quando quiser representar uma adição ou nova funcionalidade ao código. Exemplos:

feature/iniciar git

feature/estado botão comprar inativo e esgotado

Hotfix (trabalho rápido)

Iniciar suas mensagens **Git** com "**hotfix**" é utilizado para indicar a modificação de algo já existente no código, geralmente para melhorar, aprimorar ou refatorar uma funcionalidade pré-existente. Serve para corrigir algo pontualmente.

hotfix/cor botão comprar

Bugfix (correção)

O "**bugfix**" é uma mensagem direcionada à correção de erros (**bugs**) identificados no código.

Essas mensagens indicam tarefas essenciais para manter a estabilidade e a qualidade do software, solucionando problemas que podem prejudicar o funcionamento correto do sistema.

bugfix/botão comprar (cor da fonte)

O comentário "**bugfix/botão comprar (cor da fonte)**" é um exemplo específico de um tipo de correção de erro. Neste caso, foca-se na resolução de um problema específico relacionado à aparência do botão de compra, como a cor incorreta da fonte, garantindo assim o histórico de mudanças da experiência visual para o usuário final.

Maiores detalhes sobre as mudanças podem ser inclusos na área de descrição desses comentários.

SONAR LINT

Quando você solicita que um desenvolvedor execute uma tarefa específica, é importante que as modificações mantenham alto padrão de qualidade de código. Para isso, o **SonarLint** faz a auditoria da qualidade do código do programador a cada alteração do código no repositório.

A análise dinâmica, por meio do **Lint**, fornece *feedback* instantâneo enquanto você codifica. O

SonarLint destaca falhas de codificação e explica porque o problema é prejudicial e como corrigi-lo.

Vale a pena ressaltar também o uso da ferramenta **Prettier**, que formata o código do programador automaticamente ao salvar o arquivo, deixando-o mais legível, mas só de forma mais visual e não funcional, o que é muito útil.

PADRÕES DA COMUNIDADE

É fundamental seguir as boas práticas de qualquer mercado, mas é melhor ainda seguir os padrões da sociedade. Segue link para visualizar esses padrões:

<https://github.com/miltonbolonha/gatsby-theme-v5-boilerplate/community>

Assim garantimos segurança, eficiência e perfeita colaboração entre os mais diversos membros da equipe.

ROTAS

Aproveitando que estamos navegando, vamos traçar algumas rotas. Eu gosto de separar as rotas existentes em três tipos, a saber:

- Rotas Públicas

- Rotas Privadas
- Rotas Externas

Quando você acessa um website, é igual a chegar a uma repartição pública para fazer algum tipo de burocracia. Se você tem que conseguir alguma autorização do governo para trânsito, você pega a rota verde, se está em busca coisas sociais você entra na rota azul, então você chega até uma fila e entrega a sua requisição.

As **rotas públicas** são, por exemplo, as páginas web que estão à disposição do público em geral.

Rota privada é quando você tem que ter algum tipo de autorização/credencial para navegar por aquela rota, um exemplo são as redes sociais.

Por fim, as **rotas externas** são requisições em serviços de terceiros, por exemplo, quando um cliente termina de preencher e envia um formulário de contato, então o sistema terceirizado **SendGrid** envia um e-mail para a equipe, com as informações do cliente.

DICAS ÚTEIS

ADMINISTRANDO TAREFAS

Quando o programador começar um trabalho de desenvolvimento web, ele deve ter em mente que precisará ter alguns cuidados específicos, antes de começar o trabalho de fato.

Em muitos projetos de programação, é comum o programador passar mais tempo lendo uma documentação de alguma tecnologia, ou entendendo a arquitetura do código, do que programando em si.

Mantenha sempre todas as tarefas anotadas em listas. Faça e refaça essa lista. Aos poucos, você vai adquirir suas próprias técnicas de administração de tarefas. Até lá, sugerimos que use o quadro kanban.

MÉTODO ÁGIL & QUADRO KANBAN

Metodologia ágil é uma forma de conduzir projetos rápida, focada em processos e na conclusão de tarefas e subtarefas em uma esteira de produção.

O termo “Kanban” é de origem japonesa e significa “cartão” e propõe o uso de cartões (cards) para indicar e acompanhar o andamento da produção dentro da esteira. Trata-se de uma esteira de produção baseada em macro eventos, como iniciar o trabalho, trabalho em andamento, testando o trabalho, trabalho concluído; as tarefas são gerenciadas por comentários e ao serem movidas de um momento a outro dentro do processo criativo.

O uso de cartões para indicar uma tarefa, facilita a visualização e interações da equipe de produção com cada tarefa. Esse cartão conterá a história de cada esforço para determinado trabalho.

Quem decide quais cards serão criados, seus detalhes, explicações, quantidade de horas a gastar e os aninhamentos, é a gerente de projeto junto ao cliente. É interessante que dentro desse processo a gerente tenha uma liderança técnica para apresentar o trabalho para a área de programação e esse líder decidir qual pessoa programadora é mais qualificada para aquela tarefa.

Cada cartão deve conter uma tarefa apenas. Para tarefas relacionadas ou aninhadas, busque sistemas que agregam os cartões de forma inteligente. Evite um cartão contendo muitas tarefas, listas longas e diversas, o cartão deve ser bem específico e muito descritivo.

Cada cartão deve ir em uma coluna da esteira de eventos, sendo que as primeiras linhas de cada coluna

devem se destinar aos cartões urgentes e abaixo aqueles com maior importância.

ESCOPO DA TAREFA

Uma tarefa é como um cartão (*card*) ou um adesivo de recado (*post-it*) que você faz anotações para sempre se lembrar do que tem que fazer. As tarefas possuem normalmente a seguinte estrutura:

- Título
- Descrição
- Estado (status)
- Responsável
- Designado a
- Anexos
- Comentários

CONVENÇÃO DE NOMES PARA AS TAREFAS

O título da tarefa deve descrever a tarefa, com as convenções de nomeação para tarefas.

O problema é que essas convenções não existem, vamos tentar explicar a seguir algumas regrinhas lógicas que aprendemos ao redor do mundo através dos anos e podem ser muito úteis no dia-a-dia.

Vamos supor que uma agência web quer que seu colaborador, a pessoa programadora, modifique o posicionamento do botão de compra movendo-o da esquerda para direita. O web site pertence a loja fictícia “*MBuy Shop*”. Essa é a primeira tarefa que essa empresa pede para a agência. Qual será o nome da tarefa?

MBS-01: Botão Comprar (mudar posição)

Como a agência possui muitos clientes e no caso desse cliente, além do website da “*MBuy Shop*” eles também fazem os comerciais de TV, com isso o nome de uma tarefa relacionada a TV ficaria mais ou menos assim:

MBS-TV-01: Gravar comercial 2

Perceba como facilita na identificação da tarefa haver um ou mais indicadores claros no título da tarefa.

ESTEIRA DE EVENTOS DE PRODUÇÃO

Esteiras de eventos de produção são como filas e em cada momento da fila, será executada uma parte da criação do produto final.

Em diversos momentos das nossas vidas lidamos com filas de produção, seja na hora de fazer um bolo, um carro ou um casamento.

Tudo começa com o apanhado geral das tarefas, que são empilhadas na coluna que pode ser dita como um baú chamado Backlog/Pendências.

PENDÊNCIAS

O baú das tarefas se refere a uma lista de tarefas acumuladas, também conhecida como a coluna das **Pendências**.

É importante que o time inteiro trabalhe ativamente para alimentar as pendências, para que assim o produto seja sempre atualizado e todos estejam sempre de olho em como melhorar o sistema em que a equipe está trabalhando.

Os líderes devem estar atentos ao fluxo de distribuição de tarefas, prazos e a quantidade de tarefas por pessoa programadora.

É comum concentrar tarefas do mesmo cliente na mesma pessoa programadora, aproveitando-se do seu conhecimento prévio sobre a plataforma.

A FAZER

As tarefas que saem das pendências para estarem prontas para começar a serem executadas. É hora de

executar o plano. Para isso, é necessário o “OK” do cliente e também a tarefa estar atribuída a uma trabalhadora específica.

A profissional deve ler toda a tarefa, comentários e buscar toda informação que lhe faltar, além de tecer perguntas objetivas sobre o trabalho e encontrar um entendimento macro do que vai fazer.

Pessoas com déficits de concentração, ou psicossocial, podem ter problemas de entendimento de uma tarefa. Nesse caso, pode ser vantajoso fazer um esforço extra para entender a tarefa, depois largar ela para fazer depois. Essa pessoa pode querer fazer esquemas visuais, ou pedir para outras pessoas explicarem a tarefa para ela.

O gerente de projetos deve se comunicar em uma linguagem universal, para isso é necessário conhecimentos, mesmo que superficiais de programação, para esse profissional que vai criar tarefas de alteração nos códigos.

TRABALHANDO

Quando a trabalhadora entende a sua tarefa, ela está apta à mudar a tarefa para a coluna trabalhando.

Ao longo dessa execução, essa pessoa fará comentários na tarefa, documentando o seu trabalho na medida do necessário.

TESTANDO

Quando o profissional executa a mudança no código, o responsável deve testá-la. Retornar se tiver bugs, ou dar encaminhamento se a tarefa estiver finalizada.

Se o gerente de projetos devolve a tarefa para a programadora, contendo novos comentários sobre o trabalho, talvez seja necessário esse gerente ser mais descritivo com as tarefas desde o começo. Certifique-se de ser simples e ter uma plataforma que realmente gerencie o seu trabalho de forma ágil.

PARA APROVAÇÃO

Na maioria dos casos é o cliente quem aprova ou reprovava o trabalho final, que é testado em um ambiente de ensaios/testes.

Porém, é imprescindível a aprovação dos líderes. Estejam atentos a processos lentos, cheios de idas e vindas. Adaptem-se com humanidade e diálogo, visando sempre o coletivo e o bem-estar da pessoa programadora. Afinal, toda agência é feita de talentos.

IMPLEMENTAÇÃO

Testado e aprovado, é hora de colocar em produção. Nesse ponto todos os testes já foram feitos, bem como as atualizações.

É comum encontrar novas demandas, o trabalho nunca acaba. Inclua novas tarefas no backlog.

OBRIGADO

Repita a leitura deste volume quantas vezes for preciso. Pesquise muito. Você tem uma longa jornada pela frente, o mais importante é que o primeiro passo foi dado.

Seja bem-vindxs ao meu mundo, ao mundo da programação e ao Brasil beta 2-0.X.X.

NOTAS DE VERSÃO

V.1 – 03/01/24 - publicação.

V.1.1 – 30/01/24 - correção, melhoramento de sintaxe e pequenos enxertos. Nova capa.

V.1.2 – 17/04/24 - correção.

Mano Dev

Milton Bolonha

Toda esperança
É como uma criança
E todo vendaval
Ele acaba no final
Brasil beta chegará em breve
Vai Mano/Mina Dev
Caminhar na brisa leve!

*A Obra editada no dia 03/01/2024 pela Editora BOOK4DEV.
Este título é de inteira propriedade do autor. Sendo proibida a sua cópia,
distribuição ou utilização pública ou privada de qualquer espécie, sem
prévia autorização do autor.*